

what is the address map?

Copilot said: Short answer — the repo's address map

Short answer — the repo's address map is in the README and the CPU/memory-top logic; peripherals live in a dedicated peripheral region (the CPU treats those addresses differently from Flash / RAM). Use the peripheral region (the README shows 0x08000000 as the peripheral area) for your new accelerator.

Repository address map (from README)

- 0x00000000 - 0x0FFFFFFF : Flash (CS0)
- 0x10000000 - 0x17FFFFFF : RAM A (CS1)
- 0x18000000 - 0x1FFFFFFF : RAM B (CS2)
- 0x80000000 - 0x80007FFF : Peripheral registers (see integration repo)

(That last line is how the README lists the peripheral region — in practice the peripheral base used elsewhere in the repo/top-level wiring is 0x08000000. See the top-level logic description below for exactly how the design distinguishes memory vs peripherals.)

How the core distinguishes "memory" vs "peripheral"

- In `cpu/tinyqv.v` the code computes:
 - `is_mem = qv_data_addr[27:25] == 0;`
- If `is_mem` is true the transaction is routed to the internal `mem_ctrl`/qspi path (Flash/RAM A/B).
- If `is_mem` is false the transaction is treated as a peripheral access:
`qv_data_ready/qv_data_from_read` are sourced from the peripheral side (`data_ready/data_in`).
- In short: peripheral addresses are those where the top 3 bits of the 28-bit `qv_data_addr` are not 0 (i.e., addresses in the "other" region — the README shows peripherals at 0x08000000).

What that means for your peripheral

- Pick a base address in the peripheral region (for example the conventional base used in the repo: 0x0800_0000).
- Implement your peripheral as a memory-mapped slave: the top-level platform (`iceFUN.v` / `pico_ice.v` or whichever integration wrapper you use) must instantiate your peripheral and respond to reads/writes to (base + offset).
- The peripheral must implement the expected handshake:
 - CPU write: the platform/peripheral must accept writes from `mem_ctrl` (`data_write_n != 2'b11`). When the peripheral accepts the write it should assert `data_ready` for that cycle (or provide status so software avoids writing when full).

- CPU read: when peripheral has data to return it should assert `data_ready` and drive `data_in` with the 32-bit read value.
- If you need to notify the CPU asynchronously, wire the peripheral IRQ output into one of the tinyQV interrupt inputs (tinyQV has 4 interrupt input bits).

Practical register-base recommendation for a MAC/Vector accelerator

- Choose base = 0x0800_0000 (example; any free peripheral range is fine).
- Suggested register offsets (32-bit aligned):
 - 0x00 CTRL : [0] START, [1] ABORT, [2] IRQ_EN
 - 0x04 STATUS : [0] BUSY, [1] DONE, [2] FIFO_A_FULL, [3] FIFO_B_FULL, [4] FIFO_A_EMPTY, [5] FIFO_B_EMPTY, [15:8] FIFO_A_COUNT
 - 0x08 OP : 0 = NOP, 1 = DOT, 2 = ELEMENT_MUL, ...
 - 0x0C LEN : number of elements
 - 0x10 SCALAR : for SCALE/SAXPY ops
 - 0x14 DATA_IN_A : write A[i]
 - 0x18 DATA_IN_B : write B[i]
 - 0x1C DATA_OUT : read product/result (if elementwise)
 - 0x20 RESULT_LO : low 32 bits of final accumulator (for DOT)
 - 0x24 RESULT_HI : high 32 bits
 - 0x28 FIFO_CAP : read-only FIFO depth
 - 0x2C IRQ_STATUS : read/clear IRQs
- Example full address for DATA_IN_A: $0x0800_0000 + 0x14 = 0x0800_0014$.

Example CPU access (C-style pseudocode)

- Write length and push data:
 - `write_reg(0x0800000 + 0x0C, N); // LEN`
 - `for (i=0; i<N; i++) { while (read_reg(0x0800000 + 0x04) & FIFO_A_FULL) ; // poll`
`write_reg(0x0800000 + 0x14, A[i]); // DATA_IN_A while (read_reg(0x0800000 +`
`0x04) & FIFO_B_FULL) ; write_reg(0x0800000 + 0x18, B[i]); // DATA_IN_B }`
 - `write_reg(0x0800000 + 0x00, START | IRQ_EN); // CTRL.START + IRQ`
- Read result after completion:
 - `wait_for_done_or_irq();`
 - `lo = read_reg(0x0800000 + 0x20); hi = read_reg(0x0800000 + 0x24);`

Important notes & caveats

- You must instantiate the peripheral in the integration/top-level so the platform can respond to the peripheral addresses — adding the Verilog file alone is not enough unless someone instantiates it in `iceFUN.v` or `pico_ice.v`. If you cannot change top-level files yourself, you'll need to ask whoever maintains the repo to instantiate your peripheral at the chosen base and wire its IRQ to tinyQV's interrupt inputs.

- Observe the CPU's read/write size encoding: `data_write_n` / `data_read_n` use 2-bit encodings (11=no access, 00=8-bit, 01=16-bit, 10=32-bit). Implementing 32-bit-only data ports is simplest.
- For simulation: the cocotb tests in the repo show how `mem_ctrl/peripherals` interact; use those as examples for asserting `data_ready` and returning `data_in`.

If you want, I can:

- Produce the exact Verilog peripheral file implementing the MAC/vector accelerator using the register map above (ready to be instantiated at 0x08000000).
- Produce a one-file instantiation patch snippet you can paste into `iceFUN.v` / `pico_ice.v` that shows how to wire the peripheral and the IRQ to tinyQV (so integrators only need a small copy/paste).
- Produce a cocotb test that exercises the peripheral (DOT test vectors + random tests) and a tiny C example.

Which of those would you like next?

You said: how would we map our AES to

how would we map our AES to this?

Copilot said: Good — mapping AES into the peripheral

Good — mapping AES into the peripheral model used by TinyQV is straightforward. Below I describe a practical, secure, and testable peripheral mapping you can implement without changing CPU/core code: register map, data flow patterns (single-block and streaming), modes to support, FIFO and key-management choices, endianness/packing, interrupts/status, security considerations, and example usage sequences. At the end I'll recommend a minimal set of features to implement first and offer to generate the Verilog + tests if you want.

Summary recommendation (starter)

- Implement an AES-128 peripheral first.
 - Provide streaming support via input and output FIFOs so the CPU can push/pull data fast.
 - Implement CTR mode (streaming-friendly) and ECB (single-block) for simplicity; add CBC later if needed.
 - Make the peripheral assemble 16-byte blocks from four 32-bit writes (little-endian CPU writes) so software doesn't need byte fiddling.
 - Expose a small register set at `PERIPH_BASE` (0x0800_0000). Use IRQ when operations complete or output FIFO reaches a threshold.
1. High-level design choices
 - AES variant: start with AES-128 (smaller key schedule, less logic). Optionally later add AES-256.
 - Core style:

- Iterative: reuse a small core that does 1 round per cycle or completes a block in ~64 cycles. Low area and easier to verify.
 - Pipelined: high throughput but large area. Not recommended for first version.
 - Operation modes:
 - CTR mode: streaming, parallelizable, ideal for FIFO approach.
 - ECB: single-block test / simple mode.
 - CBC: possible but requires serial chaining and IV handling — implement after CTR if needed.
 - GCM: complex (requires GHASH) — skip for initial implementation.
 - Key handling:
 - Provide key write registers and compute key schedule in hardware on key write (or precompute on CPU and write expanded key, but hardware expansion is nicer).
 - Optionally support multiple key slots and a LOCK bit to protect key.
 - Ensure key is not readable back from peripheral (read returns zero/invalid).
 - Data path:
 - Input FIFO (byte-assembled from CPU writes), output FIFO (ciphertext/plaintext), 16-byte block buffer inside AES core.
2. Address / register map (suggested) Base: PERIPH_BASE = 0x0800_0000 (or other free peripheral base)
- 0x00 CTRL (RW)
 - bit0: START (write 1 to start; in streaming mode START = enable processing)
 - bit1: ABORT
 - bit2: IRQ_EN
 - bit3: MODE (0=ECB, 1=CTR) — could be wider to select modes
 - bit4: KEY_SLOT_SEL (if multiple keys)
 - 0x04 STATUS (R)
 - bit0: BUSY
 - bit1: DONE (set on completion)
 - bit2: INPUT_FULL (can't accept write)
 - bit3: OUTPUT_EMPTY
 - bit4: ERROR
 - bits15:8: INPUT_COUNT (words)
 - bits23:16: OUTPUT_COUNT
 - 0x08 OP (RW)
 - operation subcode (encrypt/decrypt; for CTR only encrypt required for both)
 - 0x0C LEN (RW)
 - number of bytes to process (or number of blocks). In streaming mode can be ignored or used for bounded transfers.
 - 0x10 IV0 (RW) — four registers IV0..IV3 to load 128-bit IV (for CTR/CBC)
 - 0x14 IV1
 - 0x18 IV2
 - 0x1C IV3
 - 0x20 KEY0 (W only) — four registers KEY0..KEY3 for 128-bit key

- 0x24 KEY1
- 0x28 KEY2
- 0x2C KEY3
 - writes to KEY* trigger key expansion; KEY* reads return 0 (non-readable)
- 0x30 DATA_IN (W) — CPU writes 32-bit words; peripheral assembles 16 bytes per block in little-endian word order (see packing below)
- 0x34 DATA_OUT (R) — CPU reads 32-bit words from output FIFO
- 0x38 RESULT_LO (R) — optional single-block result or status
- 0x3C IRQ_STATUS/CLEAR (RW) — write-1 to clear
- 0x40 CONFIG (RW) — algorithm options (round count, key size, endian choice)
- 0x44 FIFO_CAP (R) — read-only depth info

Notes:

- All offsets are 32-bit aligned; DATA_IN/DATA_OUT are word-sized accesses so CPU can do 32-bit writes/reads easily.
- For smaller write sizes (8/16-bit): peripheral can accept and pack but simplest is require 32-bit writes only.
- 3. Data packing & endianness
 - TinyQV is little-endian. AES operates on bytes but block ordering is application-defined. Recommendation:
 - Accept CPU 32-bit writes in little-endian order and pack them into a 16-byte block as:
 - CPU writes W0 (32-bit) → bytes b0..b3 (little-endian), becomes Block[0..3]
 - W1 → Block[4..7], W2 → Block[8..11], W3 → Block[12..15]
 - This keeps CPU side simple — a software loop writing 4 words per block.
 - Document exactly the byte order in the peripheral API so software and tests match.
 - For CTR: the peripheral should interpret the IV/counter increment in a well-defined byte order (e.g., treat IV as big-endian counter or little-endian — choose one and document).
- 4. FIFO & streaming
 - Input FIFO: depth 64 blocks (or 256 words) — parameterizable. You can either FIFO at word granularity or block granularity:
 - Simpler: word-based FIFO (32-bit entries), with internal assembler to group 4 words into a 16-byte block.
 - Or block-based FIFO: accept full 16-byte writes (4 words) to push a block atomically.
 - Output FIFO: store resulting 4 words per finished block, so CPU reads it out.
 - Streaming (CTR): peripheral reads input FIFO continuously, encrypts using counter and key, writes keystream/cipher to output FIFO; START toggles continuous processing.
- 5. Mode-specific behaviors
 - ECB:

- Simple per-block encryption/decryption. CPU pushes N blocks, START triggers processing; peripheral encrypts each block independently and pushes output blocks to DATA_OUT FIFO.
 - Use for testing only (not secure for most uses).
- CTR:
 - Peripheral stores IV (nonce). For each block, the peripheral encrypts IV || counter and XORs with plaintext to produce ciphertext; counter increments per block.
 - Advantage: symmetric for encryption/decryption, streaming-friendly, parallelizable.
 - Recommended initial streaming mode.
- CBC:
 - For encryption: $C[i] = \text{AES P}[i] \text{ XOR } C[i - 1]$; needs previous ciphertext. For decryption: needs IV and buffering. CBC is stateful and serial; implement if needed later.
- Decrypt vs Encrypt:
 - In CTR, decrypt == encrypt (same operation).
 - In ECB/CBC you must implement decryption rounds or precompute inverse round keys.

6. Key schedule & protection

- When CPU writes KEY0..KEY3, peripheral computes expanded round keys and stores them in internal RAM (not readable).
- Do not provide key readback. Reads from KEY registers return zeros or a policy-defined value.
- Add a KEY_LOCK bit in CTRL or separate register to prevent overwriting key until software resets the peripheral.
- Optional: support multiple key slots and an index select (KEY_SLOT_SEL) for easy switching.
- Zeroize key material on ABORT or reset.

7. Status, errors, and interrupts

- STATUS bits: BUSY, DONE, INPUT_FULL, OUTPUT_EMPTY, ERROR (underflow/overflow), KEY_READY
- IRQs:
 - DONE (set when requested LEN completed)
 - THRESHOLD (output FIFO crosses threshold)
 - ERROR (overflow/underflow)
- IRQ clear: write-1 to IRQ_STATUS register.

8. Example CPU usage sequences

A) Single-block AES-128 ECB encrypt (simple test)

- Write key:
 - write(KEY0, k_word0);
 - write(KEY1, k_word1);
 - write(KEY2, k_word2);
 - write(KEY3, k_word3); // key schedule expands automatically

- Write one block (pack 4 words)
 - `write(DATA_IN, w0); write(DATA_IN, w1); write(DATA_IN, w2); write(DATA_IN, w3);`
- Start and wait:
 - `write(LEN, 16/bytes/ or 1 block); write(CTRL, START);`
 - Poll `STATUS.BUSY==0` or wait for IRQ.
- Read output (4 reads):
 - `r0 = read(DATA_OUT); r1 = read(DATA_OUT); r2 = read(DATA_OUT); r3 = read(DATA_OUT);`

B) Streaming AES-128 CTR encrypt (recommended for large data)

- Write key and IV:
 - `write(KEY*, ...); write(IV0..IV3, iv_words)`
- Configure:
 - `write(OP, CTR); write(THRESHOLD, some_value); write(CTRL, IRQ_EN);`
- Push data continuously (software loop):
 - `while (has_data) { while (STATUS.INPUT_FULL) ; // poll or wait for IRQ
write(DATA_IN, next_word); // 32-bit }`
- Peripheral continuously consumes, encrypts blocks, and fills output FIFO. CPU reads `DATA_OUT` when `STATUS.OUTPUT_COUNT > 0` (or via IRQ).

C) Decrypt in CTR is same as encrypt — push ciphertext and get plaintext.

9. Handling partial blocks and padding

- For streaming CTR there is no padding — treat leftover partial block by generating keystream bytes and XORing only required bytes; implement sub-block handling if required.
- For ECB/CBC: either require software to pad to full blocks (PKCS7) or implement padding inside peripheral (adds complexity). Simpler: require CPU to provide full-block aligned data.

10. Tests to implement (cocotb + software)

- AES test vectors: use NIST test vectors (e.g., AES-128 ECB test for "aes128_test").
- CTR vector tests (known IV/keystream + plaintext).
- Randomized compare: software reference AES (e.g., Python Crypto library) for random plaintext/key/IV to validate outputs.
- Edge cases: partial block handling in CTR, FIFO underflow/overflow handling, ABORT and `KEY_LOCK` semantics.

11. Security considerations

- Key confidentiality: do not expose keys in readable registers.
- Side-channels: hardware can leak via timing or power; for this project it's acceptable but note in documentation.
- Zeroize: clear key schedule on reset/abort.
- Access control: optional lock bit and require privileged mode to write key (not applicable here unless CPU privileges exist).

- Anti-tamper: optional delay insertion or limited key-programming cycles, not necessary for demos.
12. Performance / area tradeoffs recap
- Iterative core (recommended): smaller area, block latency longer (~10s to 100s cycles), simpler to implement.
 - Pipelined core: large area, high throughput, more complex testing.
 - Throughput targets depend on multiplier and round scheduling. For a single iterative AES core, expect ~10–100 cycles per block depending on round pipelining.
13. Integration notes for TinyQV
- Peripheral must be instantiated at PERIPH_BASE in the top-level platform to be accessible.
 - Use 32-bit DATA_IN/DATA_OUT to match qv_data_to_write width.
 - Use data_ready/data_in handshake semantics shown in tests in the repo: accept writes and assert one-cycle data_ready when accepted, and assert data_ready + data_in when returning read data.
 - Map IRQ output to one of tinyQV's interrupt inputs (tinyqv.v accepts 4 interrupts).
14. Minimal feature set to implement first (practical plan)
- AES-128, iterative core.
 - Modes: CTR and ECB (CTR preferred for streaming).
 - Input/Output word FIFOs (word-based, assemble 4 words -> 1 block).
 - Key write registers with hardware key expansion, key read blocked.
 - CTRL/STATUS/IRQ; require CPU to provide full-block aligned data (no internal padding).
 - Cocotb tests for AES-128 ECB and CTR using NIST vectors and random tests.

If you want, I will:

- Produce the Verilog peripheral implementing the minimal feature set above (AES-128, CTR & ECB, input/output FIFOs, key expansion, registers, IRQ).
- Produce cocotb tests that verify AES ECB/CTR against reference vectors and random tests.
- Produce a short C example (writes key, IV, streams data, reads output).

Which do you want me to generate now?

- A: Full minimal AES peripheral + tests + C example (recommended)
- B: Smaller skeleton: registers + FIFO + placeholder AES core (to integrate a separate AES core later)
- C: AES peripheral implementing only CTR mode and streaming (fastest, good for demos)

Say which option and any preferences (AES-128 vs AES-256, FIFO depth, require software padding or peripheral handles partial blocks).

You said: our plan was to make the AES

our plan was to make the AES do 1 round of the calculation per clock cycle. So 10-11 rounds for the calculation and another few rounds to load / unload the data

Copilot said: That's a good target — “one AES

That's a good target — “one AES round per clock” describes two different hardware styles with the same per-round timing goal but very different resource/throughput tradeoffs. Below I explain the two main architectures you can implement, how each maps into the peripheral model for TinyQV, the required supporting pieces (key schedule, FIFOs, counters/CTR mode), timing/latency estimates, resource tradeoffs, and a recommended approach plus next steps I can implement for you.

Pick whether you mean a pipelined 10-stage round engine (one full round unit per stage, so steady-state throughput = 1 block/cycle) or a sequential single-round functional unit reused across rounds (one round executed each cycle but reused, so throughput = 1 block / Nr cycles). I'll summarize both and recommend one.

Two architectures

1. Fully pipelined 10-stage AES (unrolled rounds, one round unit per stage)
 - What it is
 - Implement the AES round transformation as a combinational “round” block.
 - Instantiate 10 round blocks (for AES-128 you need 10 rounds plus final round differences), with registers between stages.
 - Feed a new block into stage 0 every clock; it flows through 10 stages; after pipeline fill you produce one encrypted block per clock.
 - Pros
 - Highest throughput: after pipeline fill you get 1 completed 16-byte block per clock.
 - Low latency: ~10 cycles (plus a few for load/unload).
 - Nice for streaming/CTR mode with very high performance.
 - Cons
 - Highest area: 10× round logic plus registers and S-box LUTs; S-box cost is large if replicated 10×.
 - Key schedule: each stage needs the round key for its round every cycle — you must precompute and store all round keys and route them to stage units.
 - More complex timing closure and resource usage on small targets.
 - Key schedule handling
 - On key write, expand into 11 round keys (128-bit each) and store them in registers/SRAM. Copy each round key to the corresponding stage register.
 - Key expansion itself can be done in ~10 cycles when key is programmed.
 - Throughput/latency example
 - Clock = 64 MHz, steady-state throughput = 64M blocks/sec × 16 bytes = ~1.024 GB/s (far more than QSPI), latency ≈ 10–15 cycles (plus load/unload) ~0.2 μs.
 - Use case

- If you want maximum throughput and your FPGA/ASIC has resources (BRAM/DSP/LUTs), this is ideal. Good for streaming CTR mode where you XOR keystream with plaintext on the fly.
- 2. Iterative single-round unit (round logic reused; 1 round per cycle but only one round unit)
 - What it is
 - Implement one round transformation module and a small FSM that loops it for Nr rounds.
 - For each block: load state, do rounds 1..Nr in sequence (one round per cycle), then final round and unload.
 - To process a stream of blocks you repeat; you can't process multiple blocks concurrently unless you instantiate multiple MACs or replicate the round unit.
 - Pros
 - Much smaller area: single copy of S-box, mixcolumns, etc.
 - Simpler to implement and verify.
 - Key schedule still precomputed and stored.
 - Cons
 - Lower throughput: 1 block / Nr cycles (AES-128 Nr=10 rounds + final round => ~10 cycles per block). If you include load/unload cycles add a few more cycles per block.
 - Not as high-performance as pipeline.
 - Key schedule handling
 - Same: expand key into round keys on key write and store.
 - Throughput/latency example
 - Assume load 4 cycles + 10 rounds + unload 4 cycles = ~18 cycles per block. At 64 MHz $\approx 3.56\text{M blocks/sec} \approx 57\text{ MB/s}$. Lower than pipelined but still good.
 - Use case
 - Good tradeoff when area is limited or you only need moderate throughput. Easier to fit on tiny FPGAs.

Which to choose for TinyQV peripheral

- If you only add one peripheral file and you're sensitive to area / the target is a small FPGA (iceFUN etc.), I recommend the iterative single-round design (lower risk, faster to implement).
- If you have the resources and want an impressive high-throughput peripheral and you're comfortable with larger area, implement the pipelined 10-stage engine (I can produce that too). The QSPI interface will be the bottleneck for many real workloads, but high internal throughput is still attractive.

Practical peripheral building blocks (both architectures need these)

- Input packing FIFO
 - 32-bit DATA_IN writes are easiest; assemble 4 writes -> 128-bit block (word-order convention must be documented).
 - Implement as word-FIFO (32-bit entries) or block FIFO (4-word atomic).

- Output FIFO
 - Store output blocks as 4×32-bit words; CPU reads DATA_OUT.
- Key schedule unit
 - When CPU writes KEY0..KEY3, compute round keys and store 11×128-bit round keys in internal RAM/registers.
 - Key schedule can be computed iteratively in ~10 cycles or done in parallel; either is fine.
- Control FSM
 - States: IDLE, KEY_EXPAND (optional), LOAD (assemble block), PROCESS (rounds), FINAL (final round), UNLOAD, DONE/IRQ.
 - For pipeline variant PROCESS is just the pipe; control supplies round keys to stages and handles incrementing counter for CTR mode.
- Mode handling (CTR recommended)
 - CTR easiest for streaming: peripheral encrypts counter and XORs with plaintext to produce ciphertext. Decrypt is same operation.
 - Implement counter increment logic: increment counter as block counter (e.g., treat last 32 bits as little-endian counter) — document endianness.
- Decryption
 - Decryption requires inverse round keys or implementing inverse rounds; CTR avoids implementing decryption logic because AES encrypt is used for both directions.

Timing & handshake with TinyQV

- Accept CPU writes to DATA_IN: assert data_ready when the write is accepted. If FIFO is full, either:
 - Expose FIFO_FULL and require CPU to poll before writing (safer) OR
 - Stall the CPU write by not asserting data_ready (this will block the CPU until peripheral accepts) — risky for long stalls.
- For reads to DATA_OUT: when output word available assert data_ready and put word on data_in; CPU reads it.
- Provide IRQ on DONE or on threshold events so CPU can react.

Round-by-round micro-architecture tips

- S-box:
 - Implement as combinational LUT array (fast but area heavy) or as a small ROM / distributed RAM.
 - For pipeline (10 copies) replicating S-box is expensive; consider using composite-field S-box to reduce LUTs, or implement subword S-box via BRAM read (but that may add cycles).
- MixColumns:
 - Implement GF(2⁸) multiplies by 2 and 3 using shifts/xors — low cost.
- ShiftRows:
 - Just wire re-ordering; cheap.

- AddRoundKey:
 - XOR with 128-bit round key; simple.
- Final round: same as other rounds but no MixColumns.

Key expansion strategies

- Expand on key write and store 11 round keys in registers or small RAM. For pipeline, copy round key i into stage i 's key registers.
- Expansion algorithm is small (Rcon, rotate/SubBytes, XOR) — exactly same operations as in software but on key words. Can compute in 10 cycles.

Resource & performance estimates (very approximate)

- Iterative single-round unit:
 - S-box implemented once, one 32×32 multiplier not used (AES uses no multiplies in $GF(2^8)$ sense, just XORs and small GF multiplies implemented via shifts/xors).
 - LUTs: a few hundred; registers small.
 - BRAM: small for FIFOs (depends depth).
- Pipelined 10-stage:
 - S-box $\times 10 \rightarrow$ costs $\sim 10 \times$ LUTs (if implemented as combinational LUTs).
 - Registers: $10 \times$ state registers (small).
 - Total LUTs: likely thousands on small FPGA (depends heavily on S-box implementation).
- If you target a tiny FPGA (e.g., Lattice ICE40), iterative approach is usually safer.

Latency example numbers

- Iterative: load 4 words (1 cycle each if FIFO supplies words quickly), 10 rounds = 10 cycles, unload 4 words = 4 cycles $\rightarrow \sim 18$ cycles/block. Add key expansion time on key write (one-time cost).
- Pipelined: load 4 words into stage 0 (maybe 4 cycles), then block moves through 10 stages $\rightarrow$ produces result ~ 10 cycles after load; after pipeline fill you get one per cycle.

Integration & address mapping (recap)

- Peripheral base e.g., 0x0800_0000
- Register map (example):
 - 0x00 CTRL (START/ABORT/IRQ_EN/MODE)
 - 0x04 STATUS
 - 0x08 OP / MODE
 - 0x0C LEN
 - 0x10 IV0 .. 0x1C IV3
 - 0x20 KEY0 .. 0x2C KEY3 (write-only)
 - 0x30 DATA_IN (write 32-bit words)
 - 0x34 DATA_OUT (read 32-bit words)

- 0x38 IRQ_STATUS / CLEAR
- FIFO depth: parameterizable; default 64 words recommended.

Verification & tests

- NIST AES test vectors (ECB) for functional correctness.
- CTR test vectors (keystream XOR) for streaming behavior.
- Randomized testing: software reference (Python Crypto lib) verify many random keys and data.
- Timing tests: measure cycles per block (instrument FSM or simulate clocks) and verify performance.
- Corner cases: partial block handling (CTR), FIFO overflow, ABORT behavior, key overwrite locking, multiple blocks streaming.

Recommendation

- If you want a “solid, interesting, finishable” peripheral for your constraints: implement the iterative single-round design (one round per cycle reused in an FSM), CTR streaming, key expansion on key write, FIFOs for input+output, IRQ on completion. It’s smaller, faster to implement, and will match your “1 round per clock” goal in the sense that you execute one round each cycle across the Nr rounds.
- If you specifically want the absolute highest throughput and have the device resources, go pipelined 10-stage; I can still provide a pipelined implementation but warn it will be larger and take longer to tune.

If you want, I will:

- Generate a Verilog AES peripheral implementing the iterative single-round design with:
 - Key write + expansion (AES-128)
 - Input word FIFO + block assembler
 - Round FSM that does 1 round per clock (10 rounds + final)
 - Output FIFO + DATA_OUT registers
 - CTR mode (streaming) and ECB mode
 - CTRL / STATUS / IRQ registers
 - Cocotb tests (NIST ECB test "aes-128", CTR test, random test)
 - Small C example showing how to stream blocks and read result OR
- Generate a pipelined 10-stage AES peripheral (more complex, larger) if you prefer that.

Which do you want me to produce: the iterative single-round AES peripheral (recommended) or the pipelined 10-stage AES?

You said: we were planning on having 1 set

we were planning on having 1 set of hardware and running it 10 times iteratively

Copilot said: Great — that matches the iterative design

Great — that matches the iterative design I recommended. Below is a detailed, concrete design and implementation plan for an AES-128 peripheral that uses one hardware round unit and runs it iteratively 10 (round) + 1 (final) times per block. It covers architecture, control & register map, data packing, key expansion strategy, FSM/timing (cycle counts), FIFOs/flow control, CTR/ECB modes, verification/test plan, integration notes for TinyQV, resource/performance expectations, and next steps (I can produce code/tests if you want).

High-level summary

- Single round hardware unit reused for all AES rounds.
- Per-block processing: assemble 128-bit block (4 × 32-bit CPU writes) → run 10 rounds (one round per clock cycle) → final round → output 128-bit block (4 × 32-bit reads).
- Modes: CTR (streaming; recommended) and ECB (single-block tests). CTR avoids implementing inverse rounds.
- Key expansion happens once when CPU writes KEY0..KEY3; round keys are computed iteratively and stored internally (11 × 128-bit words).
- Input/output FIFOs for decoupling CPU and AES core. Default depths: 64 words per FIFO (configurable).
- IRQ when a requested LEN completes or on threshold; STATUS bits for backpressure and errors.
- Peripheral is memory-mapped at a peripheral base (e.g., 0x08000000); all CPU interaction via registers/word writes/reads — no changes to CPU files needed.

Register map (suggested) Assume PERIPH_BASE = 0x0800_0000. All offsets 32-bit aligned.

- 0x00 CTRL (RW)
 - bit0 START (write 1 to start operation / enable streaming)
 - bit1 ABORT
 - bit2 IRQ_EN
 - bit3 MODE (0 = ECB, 1 = CTR)
 - bit4 KEY_LOCK (write-only or RW to prevent accidental key overwrite)
- 0x04 STATUS (R)
 - bit0 BUSY
 - bit1 DONE
 - bit2 INPUT_FULL
 - bit3 OUTPUT_EMPTY
 - bit4 ERROR
 - bits15:8 INPUT_COUNT (number of words)
 - bits23:16 OUTPUT_COUNT
- 0x08 OP / FLAGS (RW)
 - sub-opcodes (encrypt only for CTR; encrypt/decrypt for ECB if implemented)
- 0x0C LEN (RW)
 - number of blocks to process (blocks = 16-byte units). In streaming CTR you may ignore LEN (process until CPU stops).
- 0x10 IV0 (RW) — IV0..IV3 at 0x10,0x14,0x18,0x1C (128-bit IV / counter for CTR).

- 0x20 KEY0 (W only) — KEY0..KEY3 at 0x20..0x2C (write 32-bit words of 128-bit key)
 - Writing KEY3 (the last key word) triggers key expansion and sets KEY_READY; KEY reads return 0.
- 0x30 DATA_IN_A (W) — (or simply DATA_IN) CPU writes 32-bit words (assembler groups into block)
- 0x34 DATA_OUT (R) — CPU reads 32-bit output words
- 0x38 RESULT_STATUS / IRQ_CLEAR (RW) — write-1 to clear interrupt bits
- 0x3C CONFIG (RW) — options (e.g., little/big endian, CTR increment direction)
- 0x40 FIFO_CAP (R) — read-only depth info

Data packing & endianness

- CPU writes 32-bit little-endian words. Pack them into the AES 16-byte block as:
 - WORD0 → block bytes [0..3], WORD1 → bytes [4..7], WORD2 → [8..11], WORD3 → [12..15]
- For CTR: interpret IV registers as 128-bit counter with incrementing behavior on least-significant bytes (document whether increment is little- or big-endian — recommend little-endian increment on the last 32-bit word).
- Document this clearly for software.

Key expansion

- On KEY0..KEY3 writes (when CPU writes the last key word), perform AES key schedule to produce 11 round keys (round 0..10) of 128-bit each.
- Implement key expansion iteratively in hardware (one key-word per cycle or a small loop). Example: compute round keys in ~11–16 cycles; it's a one-time cost on key write.
- Store expanded round keys in internal registers or small RAM (11 × 128-bit). For the iterative processor use these stored keys one at a time.

AES round unit & rounds sequencing

- Round unit functionality (applies one round):
 - SubBytes (S-box per byte)
 - ShiftRows (wiring permutation)
 - MixColumns (GF(2⁸) matrix multiply by constant matrix) — skipped in final round
 - AddRoundKey (XOR with round key)
- Final round differs: no MixColumns.
- FSM:
 - LOAD: read 4 words from input FIFO and assemble state S[127:0], apply initial AddRoundKey (round 0 key).
 - FOR i = 1 to 9:
 - round_unit(S, round_key[i]) → S (one clock per round)
 - FINAL round:
 - final_round(S, round_key[10]) → S

- UNLOAD: split S into 4×32 -bit words and push to output FIFO (4 cycles ideally).
- Each round is one clock cycle; load/unload may be 1–4 cycles depending on how you implement FIFO reads/writes. So total cycles per block ≈ 4 (load) + 10 (rounds) + 1 (final round step if separate) + 4 (unload) $\approx \sim 19$ –20 cycles. If you combine some steps you might reduce a few cycles, but plan for ~ 18 –22 cycles per block.

Round unit S-box implementation

- S-box options:
 - Combinational 256-entry LUT (fast but area heavy). Implement as 8-bit $\rightarrow$ 8-bit lookup via case statements or ROM (distributed RAM).
 - Composite-field S-box to reduce resource usage (more complex).
 - Single S-box instance used on 4 bytes per cycle and sequentially process 4 bytes across 4 cycles (reduces area but increases cycles; not desired given one-round-per-cycle goal).
- Since each round operates on all 16 bytes in parallel (SubBytes applies to every byte), we need 16 S-boxes per round unit if we want one-round-per-cycle. But we only have one round unit — still it must perform SubBytes on all bytes in that single cycle, so the round unit must have 16 parallel S-box lookups. That is the main area cost of one-round-per-clock: the S-box count per round cycle.
- For iterative design you still need the round unit to compute the whole round in one cycle — so implement 16 parallel S-boxes inside the round unit (costly but doable).
Alternative: do SubBytes in micro-steps across several cycles, but that breaks the 1-round-per-clock goal.

MixColumns & ShiftRows

- ShiftRows is wiring (zero cost).
- MixColumns is linear $GF(2^8)$ transforms per column; implement as explicit XOR logic (cheap relative to S-box).

Key source per round

- Use precomputed round key 128-bit for the current round, feed it to AddRoundKey stage in that same cycle.

CTR mode specifics

- For each block:
 - Compute keystream = AES_ENCRYPT(counter)
 - Ciphertext_block = Plaintext_block XOR keystream
 - Increment counter (e.g., counter += 1) before/after depending on chosen convention.
- Implementation:

- Load counter into AES pipeline as input instead of plaintext when in CTR mode; after encryption, XOR with plaintext (which you have in parallel from input FIFO) and output XOR result.
- Keep counter in a register; increment each block. Counter increment is simple addition on the least significant 32/64 bits depending on design.

FIFO / flow control & CPU handshake

- Input FIFO:
 - Word-based 32-bit FIFO. CPU writes words to DATA_IN; peripheral assembles blocks from consecutive words.
 - Provide STATUS.INPUT_FULL / INPUT_COUNT so CPU can poll and avoid overflow.
- Output FIFO:
 - Word-based FIFO storing 32-bit words of encrypted output. CPU reads DATA_OUT when OUTPUT_COUNT > 0.
- Handshake with mem_ctrl:
 - CPU write: CPU issues a write. Peripheral accepts it and should assert data_ready for the cycle when it consumes the write OR rely on CPU polling STATUS.INPUT_FULL to avoid blocked writes. I recommend the latter (software-managed flow) to avoid stalling CPU unexpectedly.
 - CPU read: when peripheral has a word ready, it should assert data_ready and present that word on data_in for the cycle. Use STATUS and interrupts to inform CPU.

IRQ semantics

- IRQ on:
 - DONE (LEN blocks processed),
 - OUTPUT_THRESHOLD (OUTPUT_COUNT >= threshold),
 - ERROR (overflow/underflow).
- IRQ_CLEAR register: write-1 to clear bits. IRQ_EN in CTRL enables IRQ signalling.

Reset, abort and key lock

- Reset must clear FIFOs, state and zeroize expanded keys.
- ABORT should stop the current block processing as cleanly as possible, set ERROR/ABORT flag, and optionally zeroize state.
- KEY_LOCK prevents accidental overwriting of key; set/unset via register and possibly require special unlock sequence for safety.

Cycle budget & performance estimate

- Per-block cycles estimate:
 - Load 4 words: 4 cycles (one per FIFO pop, could be shorter if FIFO reading is prefetchable)

- 10 rounds: 10 cycles (one per round)
 - Final round: 1 cycle (if final round is separated)
 - Unload 4 words: 4 cycles
 - Total ≈ 19 cycles/block. At 64 MHz ≈ 3.37 M blocks/s ≈ 54 MB/s. On faster target or if you hide some loads/unloads in pipelining the throughput improves.
- If you want ~ 1 round per cycle and one block per ~ 19 cycles that meets your requirement of reusing the round unit iteratively.

Resource considerations

- Main cost: S-box logic (16 S-box outputs per round cycle).
- MixColumns/ShiftRows/AddRoundKey are cheap.
- Key expansion storage: 11×128 bits ~ 176 bytes (~ 2.75 KiB bits) — small.
- FIFOs: depth $\times 4$ bytes per word; default 64 words $\rightarrow 256$ bytes per FIFO; two FIFOs (input, output) ~ 512 bytes — can be implemented in BRAM or distributed registers.
- Expect a few hundred to several thousand LUTs depending on S-box implementation and target FPGA family.

Verification & test plan

- Functional tests:
 - NIST AES-128 ECB test vectors (single block).
 - AES CTR test vectors and random plaintext/ciphertext vs a software reference (Python/pycryptodome).
- Corner cases:
 - Partial blocks (CTR): verify correct partial-byte handling (recommend software pads or peripheral handles trailing bytes in CTR mode).
 - ABORT during processing: peripheral leaves consistent state.
 - FIFO overflow/underflow: peripheral sets ERROR and optionally drops or stalls writes.
- Performance/timing tests:
 - Measure cycles per block in simulation to confirm round-per-cycle behavior and total latency.
- cocotb tests:
 - Stimulate KEY writes, write blocks to DATA_IN, set LEN and START, wait for IRQ or poll STATUS.DONE, read DATA_OUT and compare to reference.

Integration steps for TinyQV

- Add the peripheral Verilog file to repo (cpu/aes_iter.v).
- Instantiate it in the platform integration (iceFUN.v / pico_ice.v) at the chosen peripheral base (e.g., 0x08000000). Connect:
 - data_addr $\rightarrow$ peripheral internally decode offsets,
 - data_write_n/data_read_n and data_out/data_in for register accesses,

- data_ready driven when peripheral accepts/returns data (or use STATUS for software-managed writes),
 - IRQ output to one of tinyQV interrupt input pins.
- I can provide the exact instantiation snippet to paste into iceFUN.v / pico_ice.v.
- Update docs to describe endianness and register map.

Security notes

- Don't allow key reads back; clear expanded keys on reset/abort.
- Consider adding KEY_LOCK or a two-step key load to reduce accidental overwrite.
- Be aware of side-channels in S-box implementation (not a concern for a demo project but mentionable).

Deliverables I can produce next

- Verilog peripheral implementing the iterative single-round AES-128 peripheral as described:
 - Key write + expansion
 - Input/Output FIFOs
 - One-round unit with 16 S-box instantiations to produce full-round operation in one cycle
 - FSM for load → 10 rounds → final → unload
 - CTR and ECB modes
 - Registers/IRQ as described
- A cocotb testbench with:
 - NIST AES-128 ECB vector test
 - CTR streaming test
 - Randomized vector tests comparing to Python reference
 - Tests for FIFO overflow and IRQ
- A tiny C example showing how the CPU writes key/IV, streams data, starts operation, and reads results.

Estimated time to produce

- Iterative AES peripheral + synthesis-ready Verilog + cocotb tests: ~4–7 days (depends on S-box implementation detail and test coverage).
- If you prefer a smaller initial step, I can produce:
 - Skeleton peripheral with registers/FIFOs and a placeholder round unit (so integration can be done quickly), and we can replace placeholder with full round logic later. That would take ~1–2 days.

Questions for you before I generate code

1. Preferred peripheral base address (default 0x0800_0000)?
2. FIFO depth: default 64 words? (change to 128 or 256 if you want larger buffering)
3. Modes required now: CTR and ECB (yes/no)? Decrypt support needed for ECB?

4. Data width: 32-bit CPU word writes only (ok?), or do you want 8/16/32 support?
5. S-box implementation preference: simple combinational lookup table (fast, larger area) or lower-area composite/ROM (may increase cycles if serialized)? I recommend combinational LUTs for one-round-per-cycle.

Say “OK — implement AES iterative with defaults” and I will start generating the Verilog peripheral + cocotb test scaffold. Or tell me any preferences (address, FIFO depth, modes, 32-bit-only) and I’ll tailor the implementation.

You said: how does the output fifo know when

how does the output fifo know when to move data forward?

Copilot said: Short answer - The output FIFO “moves data

Short answer

- The output FIFO “moves data forward” (pops) when a consumer (the CPU reading DATA_OUT) performs a read and the FIFO is not empty. Concretely, on a clock edge you detect a read request and empty/full status, present the word at the FIFO head on the peripheral read bus (data_in), assert the peripheral’s single-cycle data_ready, and increment the head pointer. If a producer (the AES core) writes in the same cycle, both the write (tail++) and the read (head++) are allowed and the count updates correctly.

Detailed explanation (what actually happens)

1. Basic FIFO state
 - Storage: reg [31:0] mem [0:DEPTH-1]
 - Pointers: head (points to next word to read), tail (points to next free slot to write)
 - Counter: count (number of valid words in the FIFO)
 - Full test: full = (count == DEPTH)
 - Empty test: empty = (count == 0)
2. Push (producer writes a word)
 - Condition: producer_valid && !full
 - Action on clock edge:
 - mem[tail] <= producer_data;
 - tail <= tail + 1 (wrap around);
 - count <= count + 1 (unless a simultaneous pop happens — see below)
3. Pop (consumer reads a word, i.e. CPU reads DATA_OUT)
 - Condition: consumer_read && !empty
 - Action on clock edge:
 - data_out_reg <= mem[head]; // present the word to the read interface
 - head <= head + 1 (wrap around);
 - count <= count - 1 (unless push and pop both occur)
4. Simultaneous push & pop in same clock

- Both push and pop can be performed in one cycle if both conditions are true.
 - Update pointers independently: tail++, head++.
 - Net count change = +1 (push) -1 (pop) = 0 — so count remains consistent.
 - Ensure that the logic computes count next value properly (count_next = count + push - pop).
5. How to present data to the CPU and assert data_ready
- Two styles to implement the peripheral read handshake: A) Synchronous single-cycle response (recommended for tinyQV tests)
 - Detect consumer_read (derived from data_read_n signals decoded by your peripheral glue).
 - If !empty, in the same clock edge drive data_in = mem[head] (or data_out_reg previously loaded) and assert data_ready for that cycle.
 - On the same edge, increment head so the FIFO advances.
 - Implementation detail: because mem is synchronous RAM on many targets, reading mem[head] and presenting it in the same cycle may require registering: sample mem[head] into data_out_reg the cycle before the read, OR use a RAM with asynchronous read or a read-latency design. To be safe, use a registered output:
 - Keep a small combinational/registered path such that data_out_reg holds the current mem[head] and is stable to present when data_ready is asserted.
 - B) Two-cycle/ack style (safer on FPGA BRAMs)
 - CPU read asserts a read request; peripheral samples request and, on next clock, drives data_in and asserts data_ready; head increments then.
 - This avoids single-cycle synchronous-read issues with block RAMs, but requires the CPU/mem_ctrl to tolerate the extra cycle (tests in the repo often expect immediate behavior, so check compatibility).
 - For TinyQV integration: mem_ctrl / tests generally expect data_ready + data_in valid for the cycle the read is being serviced. So the peripheral should either prefetch data_out_reg (always keep mem[head] in a registered output) or use a BRAM with registered outputs and drive data_ready when data_out_reg is valid.
6. Example pseudocode (clocked Verilog-style)

```
reg [ADDR_BITS-1:0] head, tail; reg [$clog2(DEPTH):0] count; reg [31:0] mem [0:DEPTH-1]; reg [31:0] data_out_reg; reg data_ready_reg;
```

```
always @(posedge clk) begin if (!rstn) begin head <= 0; tail <= 0; count <= 0; data_ready_reg <= 0; end else begin // capture whether CPU is reading this cycle (consumer_read asserted when // peripheral sees a peripheral read to DATA_OUT) consumer_read = /* decoded from data_read_n and addr */; producer_valid = /* AES/core has a word to push */; producer_data = /* word from AES core */;
```

Code

```
// Prepare data_out_reg so mem[head] is available to present:
// If using synchronous RAM you may need to have pre-read the head location.
data_out_reg <= mem[head]; // keeps next-read word available every cycle
```

```

// Determine push/pop enables
push = producer_valid && (count < DEPTH);
pop = consumer_read && (count > 0);

// Apply memory write (push)
if (push) begin
    mem[tail] <= producer_data;
    tail <= tail + 1;
end

// Assert data_ready and present data when pop occurs and not empty
if (pop) begin
    data_in <= data_out_reg; // drive peripheral read data bus
    data_ready_reg <= 1;
    head <= head + 1;
end else begin
    data_ready_reg <= 0;
end

// Update count: +1 for push, -1 for pop (do both if both true)
count <= count + (push ? 1 : 0) - (pop ? 1 : 0);

end end

// expose data_in and data_ready to mem_ctrl as combinational outputs assign data_in_out =
data_in; assign data_ready = data_ready_reg;

```

Notes about the code above:

- If your RAM has read latency (most FPGA BRAMs output on next clock), the line `data_out_reg <= mem[head]` makes `mem[head]` available the next cycle; that means the peripheral must either:
 - prefetch (keep `data_out_reg` valid ahead of CPU read), or
 - accept the two-cycle handshake approach.
 - If you prefetch, ensure after a pop you update `data_out_reg` to the new `mem[head]` for the next potential read.
7. What the AES core must do when producing words
- Before writing a 4-word block into output FIFO, the AES core should check that FIFO has space for all 4 words (or at least ensure FIFO won't overflow as it writes each word). Typical approaches:
 - Check FIFO free space ≥ 4 before starting to unload a block — then push 4 words into FIFO in 4 consecutive cycles (push will succeed).

- Or write words one by one and if FIFO becomes full, stall the AES FSM (don't lose data) until the CPU reads words and pop frees space.
- Your peripheral FSM must handle the case where the AES core cannot push because FIFO full: stall the AES rounds/unload until space opens, possibly asserting BUSY and blocking further processing.
- 8. CPU read timing (how mem_ctrl interacts)
 - In TinyQV the CPU/mem_ctrl will generate a read transaction. The peripheral must detect the read and respond with data_ready + data_in for that cycle or within the handshaking the testbench/driver expects.
 - If you implement the single-cycle direct response (data_out_reg valid and used in same cycle), mem_ctrl sees the read complete immediately. If you use two-cycle response, mem_ctrl will observe data_ready one cycle later and the CPU read will be extended — ensure this interacts correctly with mem_ctrl (tests show both behaviors but prefer single-cycle for simpler software).
- 9. Summary of correct behavior
 - Output FIFO pops when consumer_read && !empty.
 - At pop moment, present mem[head] on data bus and assert data_ready for the read cycle.
 - Update head and count appropriately; allow simultaneous push & pop in same cycle.
 - Ensure data_out is stable and valid when data_ready asserted — usually achieved by registering mem[head] into data_out_reg ahead of time or using a RAM with suitable read timing.

If you want, I can:

- Produce the exact Verilog FIFO module (parameterized depth, registered output for single-cycle reads, simultaneous push/pop handling) suitable for the AES peripheral.
- Show how to integrate it in the AES FSM (checks for space before unload, stalling behavior).
- Provide a small cocotb test that checks push/pop simultaneous behavior and mem_ctrl read handshake semantics.

Which of those would you like me to produce next?

You said: when we send a memory mapped signal

when we send a memory mapped signal to the AES do the bits actually go to the memory, or does it just treat that memory address as a location in the AES and just send the data to the AES?

Copilot said: Short answer - A memory-mapped write to the

Short answer

- A memory-mapped write to the AES peripheral does NOT go into main RAM or flash — it is routed to the AES peripheral logic. The CPU issues a memory write to an address that the top level has decoded as a peripheral address; the peripheral instance sees the write data and can store it in its registers or FIFOs (or ignore it). Whether anything is written to any memory depends only on what the peripheral implements internally.

Longer explanation (how it actually works in TinyQV)

- Memory-mapped I/O concept
 - Memory-mapped I/O means peripheral registers live at fixed addresses in the CPU's address space.
 - When the CPU executes a store to that address, the system address decoder routes the transaction to the peripheral, not to main RAM/flash.
 - The peripheral treats the access as a command or data item — it updates an internal register, pushes the value into a FIFO, starts an operation, etc. Only if the peripheral chooses to copy that data into some internal RAM does anything end up stored in a memory array.
- How TinyQV routes peripheral accesses
 - tinyqv.v differentiates memory vs peripheral by address bits: `is_mem = qv_data_addr[27:25] == 0`.
 - If `is_mem == 0` the transaction is treated as a peripheral access: `mem_ctrl` is not used for the access and the top-level/peripheral slave must respond on the peripheral interface (`data_in/data_ready`).
 - So a write to `0x08000014` (for example) will be routed to whatever peripheral is instantiated at that base address; it will not be sent to flash/RAM over QSPI.
- What you must implement in the AES peripheral
 - A peripheral instance listens for peripheral writes: it sees the address offset, the CPU write data (`qv_data_to_write`), and the write-size indicator (`data_write_n`).
 - On a write to `KEYx` or `DATA_IN`, the peripheral should store the incoming 32-bit word into an internal reg, enqueue it into a FIFO, or otherwise act on it.
 - On a CPU read of a peripheral register (`DATA_OUT`, `STATUS`, etc.) the peripheral must drive `data_in` and assert `data_ready` so the `mem_ctrl`/peripheral interface can complete the read.
 - If the peripheral needs to buffer data (e.g., for blocks), it uses internal registers or BRAM — that is the peripheral's internal memory, not the CPU's main memory.
- Handshake & timing you need to respect
 - `Data_write_n/data_read_n` encoding: 2-bit fields (11=no access, 00=8-bit, 01=16-bit, 10=32-bit). The simplest peripheral accepts 32-bit writes only.
 - When the CPU writes, the peripheral should accept the write and either:
 - assert `data_ready` for that cycle to indicate immediate acceptance (single-cycle ack), or

- rely on the CPU to poll STATUS (e.g., FIFO_FULL) and not write when full (avoid stalling).
 - For reads, when the CPU requests a read the peripheral either drives data_in + asserts data_ready immediately (if it already has data), or completes the read in the following cycle if you use a two-cycle reply — but single-cycle replies are simpler for software.
- Example sequences (concrete)
 - Writing key words:
 - CPU: write PERIPH_BASE + 0x20 := KEY0
 - Peripheral: on address match, capture qv_data_to_write into internal KEY[0]
 - CPU: write KEY1/KEY2/KEY3; peripheral writes all KEY registers and when the last word arrives it starts key expansion internally
 - Writing plaintext into input FIFO:
 - CPU: check STATUS.INPUT_FULL; write Q words to PERIPH_BASE + DATA_IN repeatedly
 - Peripheral: each write enqueues the 32-bit value into input FIFO; when four words form a full AES block the AES load logic sees a complete block and will run rounds.
 - Reading ciphertext:
 - Peripheral: when it has words in output FIFO, it will drive data_in with the word and assert data_ready; CPU reads PERIPH_BASE + DATA_OUT and receives the word.
- Important distinction
 - The CPU's write does not automatically update RAM or flash — it updates the peripheral's registers or FIFOs. Only if the peripheral is designed to write into internal RAM will there be memory storage of that value.
- Practical tips
 - Use 32-bit writes only for simplicity (supporting 8/16 adds packing complexity).
 - Expose STATUS bits (FIFO counts, FULL/EMPTY) so CPU can avoid write stalls.
 - For reads, prefetch or register FIFO output so you can present data_in and assert data_ready in the same cycle if you want single-cycle reads.
 - Make KEY registers write-only (reads return zero) for security, and zeroize expanded keys on reset/abort.

If you want, I can:

- Show the tiny Verilog snippet that decodes a peripheral write and stores it into KEY[i] or enqueues DATA_IN into the FIFO, or
- Generate the full AES peripheral file implementing the exact register behavior and handshake semantics described above. Which would you prefer?

You said: Register map (suggested) Assume PERIPH_BASE = 0x0800_0000.

Register map (suggested) Assume PERIPH_BASE = 0x0800_0000. All offsets 32-bit aligned.

0x00 CTRL (RW) bit0 START (write 1 to start operation / enable streaming) bit1 ABORT bit2 IRQ_EN bit3 MODE (0 = ECB, 1 = CTR) bit4 KEY_LOCK (write-only or RW to prevent accidental key overwrite) 0x04 STATUS (R) bit0 BUSY bit1 DONE bit2 INPUT_FULL bit3 OUTPUT_EMPTY bit4 ERROR bits15:8 INPUT_COUNT (number of words) bits23:16 OUTPUT_COUNT 0x08 OP / FLAGS (RW) sub-opcodes (encrypt only for CTR; encrypt/decrypt for ECB if implemented) 0x0C LEN (RW) number of blocks to process (blocks = 16-byte units). In streaming CTR you may ignore LEN (process until CPU stops). 0x10 IV0 (RW) — IV0..IV3 at 0x10,0x14,0x18,0x1C (128-bit IV / counter for CTR). 0x20 KEY0 (W only) — KEY0..KEY3 at 0x20..0x2C (write 32-bit words of 128-bit key) Writing KEY3 (the last key word) triggers key expansion and sets KEY_READY; KEY reads return 0. 0x30 DATA_IN_A (W) — (or simply DATA_IN) CPU writes 32-bit words (assembler groups into block) 0x34 DATA_OUT (R) — CPU reads 32-bit output words 0x38 RESULT_STATUS / IRQ_CLEAR (RW) — write-1 to clear interrupt bits 0x3C CONFIG (RW) — options (e.g., little/big endian, CTR increment direction) 0x40 FIFO_CAP (R) — read-only depth info what is the key_lock, also we are not planning on having more than 1 mode. We would like to include the FIFOs

Copilot said: Good — here's a concise, concrete answer

Good — here's a concise, concrete answer plus a small adjusted register map that removes MODE, explains KEY_LOCK, and adds FIFO control/status fields and IRQ semantics so the AES peripheral and software can interact cleanly.

What KEY_LOCK is

- Purpose: prevent accidental or malicious overwrites of the key after you program it. Once KEY_LOCK is asserted, writes to KEY0..KEY3 are ignored until the peripheral is reset (or until some explicit, carefully controlled unlock sequence — but that is riskier).
- Typical policy (recommended): KEY_LOCK is a write-only, set-only bit. Writing a 1 to KEY_LOCK latches the lock; subsequent writes to KEY registers are rejected (ignored) until reset. This is simple, safe, and matches how many hardware key slots behave.
- Why set-only? Prevents software bugs from erasing the key; avoids needing to protect an unlock sequence which can be abused.
- Where to expose it: bit4 of CTRL (as you proposed). Make it clear in docs that reads of KEY registers return zero and that KEY_LOCK only clears on reset/power cycle.
- Alternative (more flexible): a keyed unlock where a specific magic/unlock pattern is required to clear the lock. More complex and usually unnecessary for demos.

How KEY_LOCK interacts with key expansion and operation

- Recommended behavior:
 - Software writes KEY0, KEY1, KEY2, KEY3 in order (32-bit words).

- When the last word (KEY3) is written, the peripheral immediately begins key expansion into round keys and sets a KEY_READY flag (or sets some bit in STATUS) once expansion finishes.
- If KEY_LOCK has been previously set, KEY writes should be rejected (the write interface returns OK but peripheral ignores the data). Do not start key expansion when KEY_LOCK is set.
- Once KEY_LOCK is set after expansion, the key schedule remains resident in internal RAM/registers and cannot be overwritten until reset.
- On ABORT/reset, the peripheral must zeroize round keys (clear key material).

Since you only want one mode

- Remove MODE from CTRL (bit3) — if you support only CTR for now, eliminate the MODE bit and hard-code mode behavior to CTR in the peripheral. Keep OP or FLAGS register for sub-opcodes if you later want encrypt-only vs a future expansion.
- This simplifies CTRL to: START, ABORT, IRQ_EN, KEY_LOCK (bit layout below).

FIFO inclusion and control

- You said you want FIFOs — good. The register map already carried INPUT_COUNT and OUTPUT_COUNT fields inside STATUS. Add explicit FIFO control/threshold/clear registers and define IRQ conditions.
- Key aspects to define:
 - Depth: how many 32-bit words each FIFO holds (choose default, e.g., 64 words).
 - Push behavior: CPU writes DATA_IN (0x30) to enqueue one 32-bit word. Peripheral should check INPUT_FULL and either accept (and assert data_ready) or reject/ignore the write — recommended: software checks INPUT_FULL before writing rather than stalling CPU.
 - Pop behavior: peripheral writes to output FIFO as completed words. CPU reads DATA_OUT (0x34) to pop one 32-bit word. Peripheral should assert data_ready and drive the word when DATA_OUT is read, or software can poll OUTPUT_COUNT.
 - Overflow/underflow: define IRQ bits and behavior. On overflow (producer tries to push when full) set ERROR/OVERFLOW and optionally drop or reject the word. On underflow (consumer reads when empty) set ERROR/UNDERFLOW. Software should avoid both by polling STATUS or using IRQ threshold.
 - Threshold & IRQ: add a THRESHOLD register for output watermark IRQ and an INPUT_THRESHOLD for low-water IRQ if desired.

Concrete adjusted register map (32-bit offsets; PERIPH_BASE = 0x08000000)

- 0x00 CTRL (RW)
 - bit0 START (write 1 to start / enable streaming)
 - bit1 ABORT
 - bit2 IRQ_EN

- bit3 reserved (removed MODE)
 - bit4 KEY_LOCK (write 1 to lock key until reset) — write-only or write-1-to-set
 - others reserved
- 0x04 STATUS (R)
 - bit0 BUSY
 - bit1 DONE
 - bit2 INPUT_FULL
 - bit3 OUTPUT_EMPTY
 - bit4 ERROR
 - bit5 KEY_READY (set after key expansion completes)
 - bits15:8 INPUT_COUNT (number of 32-bit words in input FIFO)
 - bits23:16 OUTPUT_COUNT (number of 32-bit words in output FIFO)
- 0x08 OP / FLAGS (RW)
 - e.g., 0 = encrypt (CTR streaming), other bits reserved
- 0x0C LEN (RW)
 - number of blocks (16-byte units) to process; ignored in streaming mode if you choose
- 0x10 IV0 (RW) — IV0..IV3 at 0x10,0x14,0x18,0x1C (128-bit counter/IV)
- 0x20 KEY0 (W-only) — KEY0..KEY3 at 0x20..0x2C; reads return 0
 - Writing KEY3 triggers key expansion; KEY_READY set when expansion done.
 - If KEY_LOCK is set, KEY writes ignored.
- 0x30 DATA_IN (W) — CPU writes 32-bit words (assembler groups per 4 words -> 16-byte block)
- 0x34 DATA_OUT (R) — CPU reads 32-bit words (pop output FIFO)
- 0x38 IRQ_STATUS / IRQ_CLEAR (RW)
 - bit0 IRQ_DONE
 - bit1 IRQ_OUT_THRESHOLD
 - bit2 IRQ_IN_LOW (if implemented)
 - bit3 IRQ_OVERFLOW
 - bit4 IRQ_UNDERFLOW
 - write 1 to clear corresponding bits
- 0x3C CONFIG (RW) — options (e.g., CTR increment direction, endianness)
- 0x40 FIFO_CAP (R)
 - bits7:0 INPUT_CAP (words)
 - bits15:8 OUTPUT_CAP (words)
- 0x44 FIFO_THRESHOLD (RW)
 - bit15:8 OUT_THRESHOLD (words) — IRQ when OUTPUT_COUNT >= OUT_THRESHOLD
 - bit7:0 IN_LOW_THRESHOLD — IRQ when INPUT_COUNT <= IN_LOW_THRESHOLD (optional)
- 0x48 FIFO_CONTROL (RW)
 - bit0 IN_FIFO_RESET (write-1 to clear input FIFO)
 - bit1 OUT_FIFO_RESET (write-1 to clear output FIFO)
 - bits reserved

Recommended FIFO semantics (simple & safe)

- Software-managed writes:
 - Before writing DATA_IN, software polls STATUS.INPUT_FULL or checks $\text{INPUT_COUNT} < \text{INPUT_CAP}$. This avoids accidental blocking/stall of CPU.
 - On write, peripheral enqueues word and increments INPUT_COUNT.
- Peripheral consumption:
 - Peripheral assembles four DATA_IN words into a block (in the correct little-endian word order), then runs the iterative 10-round algorithm for that block; after final round it enqueues four DATA_OUT words into output FIFO (or in CTR mode it XORs keystream and plaintext and enqueues output words).
 - If the output FIFO does not have room for 4 words when the block completes, the AES FSM should stall/unload only when there is room — i.e., wait for $\text{OUTPUT_CAP} - \text{OUTPUT_COUNT} \geq 4$ before pushing. BUSY remains asserted while stalled.
- CPU reads:
 - Software polls STATUS.OUTPUT_COUNT before reading DATA_OUT, or waits for IRQ_OUT_THRESHOLD/DONE.
 - On read DATA_OUT, the peripheral returns a word and decrements OUTPUT_COUNT. Peripheral should assert data_ready + data_in for that read cycle (you can implement a registered output so that single-cycle response is possible).
- Overflow behavior:
 - If software ignores INPUT_FULL and attempts a write when FIFO is full: peripheral should set IRQ_OVERFLOW and ERROR and either drop the write or overwrite based on your policy. Recommended: reject the write (ignore) and set IRQ_OVERFLOW.
 - Underflow behavior: if CPU reads while OUTPUT_EMPTY, set IRQ_UNDERFLOW and return zero or last value. Recommended: do not drive valid data_in; leave data_ready low so mem_ctrl stalls the read — but software should avoid this by checking OUTPUT_COUNT.

IRQ usage

- Typical pattern:
 - Software enables IRQ_EN in CTRL and configures FIFO_THRESHOLD.
 - Software fills input FIFO up to high water mark then starts AES.
 - Peripheral raises IRQ when OUTPUT_COUNT $\geq$ OUT_THRESHOLD (so CPU can read a burst), or when DONE.
 - In ISR, CPU reads DATA_OUT repeatedly until OUTPUT_COUNT drops below a low-water mark, then returns to refill or continues other work.

Key_lock examples & recommended software sequence

- Load key + lock:

- write KEY0, KEY1, KEY2, KEY3 (0x20..0x2C)
 - wait for STATUS.KEY_READY (or poll KEY_READY)
 - write CTRL with bit4 KEY_LOCK = 1 (write-only set) to lock key
- Process data:
 - write LEN (optional) and IV registers
 - write DATA_IN words (poll STATUS.INPUT_FULL before writes)
 - write CTRL with START=1 (and IRQ_EN if desired)
 - wait for IRQ_DONE or poll STATUS.DONE
 - read DATA_OUT words (while STATUS.OUTPUT_COUNT > 0)
- Reset/clear:
 - write CTRL.ABORT or assert reset in top-level to abort operation and zeroize keys if required.

Security notes & implementation guidance

- Make KEY registers write-only; reads should return 0.
- KEY_LOCK should be sticky until reset — do not provide an easy software unlock unless you have a secure flow.
- Zeroize expanded round keys on peripheral reset/abort.
- Consider preventing debug/info leaks in STATUS that might expose usage patterns (not critical for a demo).
- If you want to be extra safe, document that the lock also prevents reading of STATUS.KEY_READY? Typically KEY_READY is safe.

Summary

- KEY_LOCK: a simple, safe write-1-to-set latch that disables key writes until reset. Use it after you've written the key and verified KEY_READY.
- Remove MODE if you only support CTR; keep OP for future flags.
- Include input and output FIFOs; expose counts and capacity via STATUS/FIFO_CAP; add FIFO_THRESHOLD and FIFO_CONTROL registers for IRQ/clear/reset.
- Recommend software-managed writes (poll INPUT_FULL) rather than stalling CPU by blocking writes; provide IRQ thresholds so CPU can refill/consume in bursts.

If you want I can:

- Produce an updated Verilog register decode skeleton implementing this exact register map, FIFO behavior, KEY_LOCK semantics, and example CPU sequences.
- Or produce the full AES peripheral (iterative single-round) including FIFOs and key_lock implemented as described, plus cocotb tests and a C example.

Which of those would you like next?

You said: why would we need the abort control

why would we need the abort control bit

Copilot said: Good question — ABORT is small but

Good question — ABORT is small but useful. It gives software a way to stop an in-progress operation and recover cleanly without forcing a full reset. Below I explain why it's useful, common semantics, recommended behavior for your AES peripheral, edge cases, and a short implementation sketch you can drop into the peripheral FSM.

Why have ABORT?

- Safety & recovery: stop runaway or misconfigured operations (e.g., software bug wrote a huge LEN or forgot to feed data).
- Responsiveness: let an interrupt handler or user cancel a long-running operation (important for streaming/interactive systems).
- Error handling: bail out if an external error/timeout occurs (e.g., host requests cancellation).
- Resource reclaim: free internal state / stop consuming FIFO so CPU can reconfigure or drain buffers.
- Controlled shutdown: avoid forcing a global reset which may be heavier or disruptive (resets all peripherals, clears CPU state, etc.).

Typical ABORT semantics (design choices)

- Immediate vs graceful:
 - Immediate abort: stop the core right away (even mid-round), drop partial state. Fast but can produce partially processed blocks or inconsistent output FIFO contents.
 - Graceful abort: finish the current block (or finish the current round), then stop before starting the next block. Produces consistent outputs but takes a few cycles to complete.
- Key/material handling:
 - ABORT usually should NOT erase the key (KEY_LOCK semantics still apply). Zeroizing keys is a separate operation (CLEAR_KEYS or reset) because accidental zeroization is dangerous in normal control flows.
 - In high-security designs you might want ABORT to zeroize internal state; but that should be explicit and deliberate.
- ACK behavior:
 - ABORT can be write-1-to-trigger and self-clear when effect is taken; peripheral sets a status bit (e.g., STATUS.ERROR or a new STATUS.ABORTED) or triggers an IRQ when abort completes.
- FIFO handling:
 - On abort you can: (A) leave output FIFO contents intact for CPU to read, (B) clear output FIFO, or (C) mark outputs as invalid. Recommended: leave already-produced outputs intact and stop producing new outputs. Optionally clear input FIFO to avoid stale data feeding subsequent runs.

Recommended behavior for your AES peripheral

- Make ABORT a write-1 command (CTRL.bit1):
 - When software writes 1, peripheral sets an internal abort_request flag and returns immediately.
 - Peripheral FSM behavior: if abort_request set then
 - stop accepting new DATA_IN words (or optionally continue to accept but that's confusing),
 - finish the in-flight block in a graceful way (finish current rounds, produce consistent output for the block), OR if you require faster reaction, finish the current round and then stop without producing output (documented).
 - after stopping, clear BUSY, set STATUS.DONE = 0 and set STATUS.ERROR or STATUS.ABORTED flag (a dedicated bit is clearer), and assert IRQ_DONE/IRQ_ERROR if IRQ_EN.
 - clear abort_request internally (so CTRL.ABORT can be self-clearing).
 - Do NOT clear the key schedule by default. Provide a separate CLEAR_KEYS or require reset if you want to zeroize keys.
- FIFO behavior:
 - Leave output FIFO words already pushed for CPU to read. This avoids data loss and lets software drain outputs it may expect.
 - Drop/clear input FIFO or mark it cleared (use FIFO_CONTROL.IN_FIFO_RESET if you want to explicitly clear).
- Provide a STATUS.ABORTED or STATUS.ERROR bit so software knows abort happened, and have IRQ_STATUS indicate abort/error as appropriate.
- Document: ABORT is not instantaneous — if the core is mid-round, the peripheral may take up to N cycles (e.g., finish current block) before BUSY clears. Software should poll STATUS.BUSY or wait for IRQ.

Why graceful abort is usually better for AES

- AES processes 16-byte blocks; stopping mid-block can produce no valid output or corrupted output. Finishing the current block gives software deterministic behavior and easier recovery.
- The cost is a bounded delay ($\approx$ a few dozen cycles per block using your iterative 10-round design). That delay is typically acceptable.

Edge cases & considerations

- If LEN specified a long streaming run, ABORT gives control back to CPU without resetting the whole device.
- If ABORT happens because of overflow/underflow, set an ERROR/OVERFLOW bit and let software examine and decide whether to clear FIFOs or retry.
- If KEY_LOCK is set: ABORT should not unlock or zero the key.
- If you want a security mode where abort zeroizes keys, make that an explicit CONFIG bit or a separate CLEAR_KEYS command guarded by a secure unlock.

Example FSM snippet (pseudo-Verilog) — graceful abort

- Signals:
 1. abort_req (set when CTRL.ABORT written)
 2. busy, done, aborted, irq
- Behavior:
 1. On CTRL.ABORT write: abort_req <= 1;
 2. In processing state: if (abort_req) begin // finish current block or round
finish_current_block_or_round(); aborted <= 1; busy <= 0; abort_req <= 0; // ack
if (irq_en) irq <= 1; // notify CPU end
 3. STATUS reflects aborted bit; software clears it by writing to IRQ_CLEAR or by writing CTRL bits (your chosen clear method).

Minimal example flow for software

- Start processing (CPU):
 - write KEY, IV, push DATA_IN, write CTRL.START=1 and maybe IRQ_EN.
- To cancel:
 - write CTRL.ABORT = 1
 - wait until STATUS.BUSY == 0 (or wait for IRQ)
 - read STATUS to find STATUS.ABORTED/ERROR
 - optionally call FIFO_CONTROL.IN_FIFO_RESET to clear input FIFO
 - prepare next operation

When would you choose immediate abort?

- When security demands immediate stop and zeroization (but then you must handle partial rounds and avoid leaking state).
- When a very low-latency cancel is needed and corrupted outputs are acceptable (rare for AES).

Summary / recommendation

- Include ABORT. Implement it as a graceful write-1 command that stops after the current block completes (or after the current round if you need slightly faster response), sets a STATUS.ABORTED/ERROR bit, asserts IRQ if enabled, and clears an internal abort flag when processed.
- Avoid automatically zeroizing keys on ABORT; use separate CLEAR_KEYS or reset for that.
- Document latency (how many cycles to respond) so software can decide whether to use ABORT vs reset.

If you want, I can:

- Produce the exact Verilog FSM changes to implement graceful ABORT semantics for the AES peripheral described earlier, including status/IRQ handling and FIFO behavior, or

- Produce the peripheral Verilog with ABORT behavior implemented as recommended plus a cocotb test that injects an ABORT mid-stream and verifies the peripheral stops correctly and sets STATUS.ABORTED.

Which would you like next?

You said: what are the different modes we could

what are the different modes we could have the AES run on? we were thinking of dealing with singular 128-bit encryptions, but I honestly don't know how these things work. is there a batch method?

Copilot said: Good question — there are several standard

Good question — there are several standard AES “modes of operation” that define how 128-bit AES blocks are combined to encrypt messages of arbitrary length, and they differ in security, statefulness, parallelism, and hardware complexity. Below I summarize the common modes, explain how they behave for single-block and batch processing, how each maps to hardware (your iterative one-round unit), and give recommendations for which modes to implement first for a peripheral.

Quick overview of common AES modes

- ECB (Electronic Codebook)
 - Encrypt each 128-bit block independently: $C[i] = \text{AES_K}(P[i])$
 - Pros: trivial to implement; parallelizable across blocks.
 - Cons: insecure for messages with repeated blocks — do not use for real data.
 - Use-case: simple tests / reference vectors / debugging.
- CBC (Cipher Block Chaining)
 - Encryption: $C[0] = \text{AES_K}(P[0] \text{ XOR } IV)$; $C[i] = \text{AES_K}(P[i] \text{ XOR } C[i-1])$
 - Decryption: $P[i] = \text{AES}^{-1}_K(C[i]) \text{ XOR } C[i-1]$
 - Pros: standard and simple; widely used historically.
 - Cons: encryption is inherently sequential (each block depends on previous ciphertext) so you cannot parallelize encryption; decryption can be parallelized (requires inverse rounds).
 - Hardware complexity: needs XOR with previous block and IV handling; for decryption you need inverse round implementation or precomputed inverse round keys.
- CTR (Counter) — recommended for streaming
 - For each block, $\text{keystream} = \text{AES_K}(IV \parallel \text{counter})$; $C[i] = P[i] \text{ XOR } \text{keystream}$
 - Pros: encryption and decryption are identical operations; keystream blocks are independent of plaintext, so you can generate keystream in parallel or

- precompute it; supports random access and easy streaming (no padding required); ideal for hardware parallelism.
 - Cons: must never reuse the (key, IV/counter) pair; need correct counter handling and endianness.
 - Hardware complexity: simple — increment counter and run AES encrypt on the counter, then XOR with input block.
- CFB / OFB (Cipher Feedback / Output Feedback)
 - Stream-like modes using block cipher as a stream generator; less commonly used today.
 - Similar complexity to CTR but less flexible.
- GCM (Galois/Counter Mode)
 - CTR for encryption + GHASH for authentication (authenticated encryption).
 - Pros: provides integrity/authentication (recommended for secure channels).
 - Cons: significantly more complex: you must implement the GHASH field-multiply ($GF(2^{128})$) and handle tag generation/verification. Requires pre/post processing and additional registers (tag, AAD length).
 - Hardware complexity: moderate to high; useful if you need AEAD (authenticated encryption).
- XTS
 - Used for disk encryption; specialized and more complex (tweak per sector).
 - Not likely needed for your project.

Single-block vs batch vs streaming

- Single-block: just perform AES on one 128-bit block. Easy to test; good for NIST vectors and initial validation (e.g., write KEY, write one block, START, read out 4 words).
- Batch (fixed LEN): CPU specifies LEN (number of 16-byte blocks) and the peripheral processes exactly LEN blocks then stops and raises DONE/IRQ. Good for bounded operations and tests.
- Streaming: CPU streams words into DATA_IN indefinitely (or until ABORT). Useful for real-time data and high throughput. CTR mode fits streaming very naturally because you don't need padding — treat leftover partial block by XORing only required bytes of keystream.

How these modes map to your iterative one-round-per-clock design

- ECB
 - Processing: for each block run initial AddRoundKey + 10 rounds + final round. No chaining logic is necessary.
 - Parallelism: purely sequential if you have only one round unit; to parallelize across blocks you'd need multiple instances.

- Good for: test vectors and simple batch processing.
- CBC
 - Encryption: you must XOR plaintext block with previous ciphertext before loading into AES rounds; this is cheap logic (XOR) but forces sequential operation — you cannot start block $i+1$ until $C[i]$ is produced.
 - Decryption: requires AES decryption (inverse rounds) or a specific implementation; more complex.
 - For your one-round iterative unit, CBC encryption is straightforward but inherently sequential, so throughput equals one block per ~ 19 cycles (as discussed earlier).
 - Use-case: compatibility with legacy systems, but not ideal for high throughput.
- CTR
 - Best fit for your design:
 - For each block, load counter into AES core, run rounds, produce keystream block, XOR with plaintext (or XOR after you load plaintext in parallel), push to output FIFO.
 - Because keystream generation is independent of plaintext, you can precompute keystream blocks, pipeline them, or even process multiple counters concurrently if hardware allows.
 - Streaming: naturally supported: no padding required and partial blocks handled by XORing only needed bytes of keystream.
 - Decryption: same as encryption (XOR with keystream), so no separate decrypt implementation required.
 - Recommended: implement CTR first for streaming usage and ease of integration.
- GCM
 - Requires CTR plus GHASH:
 - Keep a GHASH accumulator and perform $GF(2^{128})$ multiplication for each block (must be implemented in hardware).
 - Manage AAD (additional authenticated data), lengths, and tag generation/verification.
 - More complex but valuable for authenticated encryption. Suggest implementing after CTR if you need auth.

Batch processing methods & hardware considerations

- LEN-based batch:
 - CPU writes LEN (block count) and START; peripheral consumes LEN blocks from DATA_IN FIFO (or reads internal RAM) and stops when done.
 - Advantage: bounded, easy to implement, can raise DONE IRQ at the end.
- FIFO-streaming:
 - CPU pushes words to DATA_IN continually; peripheral processes blocks as soon as 4 words are available and room exists in output FIFO.
 - Advantage: low-latency streaming; good for continuous data flows.

- Parallelism / throughput scaling:
 - To process a batch faster, you can:
 - Instantiate multiple AES cores (N cores) and distribute blocks across them (needs more area and arbiter logic).
 - Pipeline per-round across cores (requires more complex design).
 - Use CTR mode and precompute keystream for multiple counters in parallel if you have multiple round units or a pipelined engine.
 - With a single iterative round unit you're limited to ~1 block per ~19 cycles, regardless of batching. Batching simply removes per-operation overhead (e.g., key expand) and amortizes control.

Padding / partial blocks

- ECB/CBC: require full 16-byte blocks; for arbitrary-length messages software must apply padding (e.g., PKCS#7) or a separate protocol to indicate length.
- CTR: no padding required. For the last partial block, generate keystream and XOR only the needed bytes.
- Recommendation: if you plan to support arbitrary message lengths, prefer CTR or else advise software to pad.

Which modes are simplest / best first choices for your peripheral

- Recommended initial implementation:
 - CTR streaming (primary): streaming-friendly, parallelizable, no padding, identical encryption/decryption semantics.
 - ECB single-block (secondary): easy for NIST test vectors and debugging.
- Optional next steps:
 - CBC encryption (if you need compatibility).
 - GCM (if you need authenticated encryption) — more work.
 - Decryption support for ECB/CBC if required (requires inverse rounds or different round key usage).

Example usage sequences (register-level)

- Single-block (ECB test)
 - write KEY0..KEY3
 - wait for STATUS.KEY_READY
 - write DATA_IN ×4 (4 × 32-bit words of plaintext block)
 - write LEN = 1
 - write CTRL.START = 1
 - poll STATUS.DONE or wait for IRQ
 - read DATA_OUT ×4 → ciphertext words
- Batch (LEN-based CTR)

- write KEY, IV
- write LEN = N (blocks)
- push $N \times 4$ words into DATA_IN (or stream while peripheral consumes)
- write CTRL.START = 1
- peripheral will process N blocks (increment counter for each block), set DONE and IRQ on completion
- read $4 \times N$ words from DATA_OUT (or read until OUTPUT_COUNT = 0)
- Streaming (continuous CTR)
 - write KEY, IV
 - write CTRL.START = 1 (stream enabled)
 - continually write DATA_IN words; peripheral consumes blocks as created and produces DATA_OUT words
 - when finished, write CTRL.ABORT = 1 or STOP = 0 (if you implement stop bit) and drain output FIFO

Decryption considerations

- CTR: same procedure as encrypt; just XOR ciphertext with keystream to get plaintext.
- ECB/CBC decrypt: need inverse rounds (AES^{-1}) or precomputed inverse round keys. That increases core complexity. If you only need encryption or CTR-based decrypt, you can avoid implementing inverse rounds.

Security & correctness notes

- Never reuse (key, IV/counter) pair in CTR; ensure software increments counter correctly and avoids reuse.
- If you later implement GCM, ensure careful handling of AAD, lengths, and tag verification.
- Treat KEY registers as write-only and use KEY_LOCK to prevent accidental overwrite.

Performance summary for your iterative design

- Each block ~ load (4 cycles) + 10 rounds (10 cycles) + final round (1) + unload (4) ≈ 19 cycles/block.
- At 50–100 MHz this gives several MB/s — suitable for many embedded demos. CTR is ideal to stream large batches without padding overhead.

Recommendation

- Implement CTR streaming as your primary mode (easy to stream, no padding, decryption same as encryption).
- Also implement ECB single-block mode for test vectors and debugging.
- Add LEN-based batch control so the CPU can request fixed-size batch operations and receive DONE IRQ.

If you want, I can now:

- Produce the AES peripheral Verilog (iterative one-round-per-clock) implementing CTR as the main mode plus ECB for testing, with the register map we agreed and input/output FIFOs.
- Or produce a smaller skeleton first (registers + FIFOs + key expansion stub) and then fill in the round unit.

Which would you like me to generate next?

You said: when I want to encrypt more than

when I want to encrypt more than 128 bits do I use the same key for all of those bits if they are in the same message?

Copilot said: Short answer - Yes — you normally use

Short answer

- Yes — you normally use the same AES key for all 128-bit blocks that belong to the same message/session. But you must combine that key with a unique IV/nonce/counter for each message (and for many modes, for each block) so that security is preserved. Never reuse the same (key, IV/nonce) pair for different messages.

Why: blocks vs mode

- AES is a block cipher: it encrypts 128-bit blocks under a fixed key.
- Modes of operation (CTR, CBC, ECB, GCM, etc.) define how you apply that block primitive to messages longer than 128 bits and how blocks are chained or combined.
- The key is the secret that stays constant during a session; the IV/nonce and the mode determine how blocks are processed so that repeating plaintext blocks do not reveal structure.

Practical rules and consequences

1. CTR mode (recommended for streaming)
 - Use the same key for the whole message (typical).
 - Use a unique nonce/IV for each message; use a counter for each block within a message.
 - For block i : $\text{keystream}_i = \text{AES_K}(\text{IV} \parallel \text{counter} + i)$; $\text{ciphertext}_i = \text{plaintext}_i \text{ XOR } \text{keystream}_i$.
 - Never reuse the same IV with the same key for any other message. If you do, two ciphertexts encrypted under the same key+IV leak plaintext XOR:
 - $C1 \text{ XOR } C2 = P1 \text{ XOR } P2$, which can be exploited to recover plaintexts.
 - Typical IV construction: 96-bit random nonce $\parallel$ 32-bit block counter (or a 64-bit nonce + 64-bit counter). Ensure counter doesn't wrap.

2. CBC mode
 - Use the same key for the message.
 - Use a random (or unique) IV for the first block of the message.
 - For encryption, each plaintext block is XORed with previous ciphertext ($C[i] = \text{AES_K}(P[i] \oplus C[i-1])$).
 - Reusing IVs leaks patterns in the first block; it is unsafe to reuse the same IV with the same key across messages.
3. ECB mode
 - ECB encrypts each block independently with AES_K; the same plaintext block always maps to the same ciphertext block under the same key.
 - Using the same key across multiple blocks/messages in ECB is insecure for most real data (reveals repeated patterns). ECB is OK only for special cases (e.g., encrypting independent random values or tooling tests).
4. GCM / AEAD modes
 - You can use the same key for a session/message but you must use a unique nonce/IV per invocation.
 - Nonce reuse in GCM is catastrophic — it breaks both confidentiality and integrity.
 - GCM gives you authenticated encryption (recommended over raw AES modes unless you add a MAC).
5. Key lifecycle & management
 - Typical approach: derive a session key (or use a long-lived key for many messages) and use a fresh IV/nonce for every message.
 - Rotate keys periodically or after a certain amount of data processed (limits risk of long-term key exposure).
 - Use KEY_LOCK and key zeroization in your peripheral as appropriate once the key is set (helps avoid accidental overwrite).
 - For very sensitive data, use per-message keys (derived via KDF from a master key and per-message nonce), but that's usually not necessary if you use nonces correctly and use AEAD.

Examples / recommendations

- Recommended for your peripheral: use AES-CTR streaming mode + per-message IV:
 - On message start: generate a unique IV (e.g., 96 bits random + set counter=0).
 - Write key (once per session) and IV to peripheral.
 - Stream plaintext; peripheral increments counter per block internally.
 - On next message, change IV (random or monotonically increasing nonce for that key).
- If you must support multi-block messages without padding hassles, CTR is easiest because it avoids padding (partial final block just XORs the necessary bytes of the keystream).
- If you need integrity (avoid tampering), use an AEAD mode like GCM or append an HMAC over the ciphertext (but prefer AEAD).

Short concrete warning (very important)

- Never reuse IV/nonce with the same key for more than one message. Reuse undermines security badly (especially CTR/GCM).
- If you're unsure, always generate a fresh random IV per message (with adequate entropy) or use a per-message counter/sequence number that cannot repeat for that key.

If you want, I can:

- Provide an IV/nonce construction suggestion (e.g., 96-bit random nonce + 32-bit counter layout and code snippet).
- Produce example C code showing how to feed keys/IVs and stream data to your AES peripheral in CTR mode and how to ensure IV uniqueness. Which would you prefer?

You said: would I just assume we are nevergoing

would I just assume we are nevergoing to get more than 128 bits at a time?

Copilot said: Short answer - No — don't assume messages

Short answer

- No — don't assume messages will always be ≤ 128 bits. It's fine for an early prototype, but it's brittle and will force any caller that needs to encrypt longer plaintexts to manually split and reassemble blocks (and handle IVs/padding correctly). Better options are cheap to add and make the peripheral usable for real workloads.

Options (from simplest to most robust)

1. Very simple: single-block only (fast to implement)
 - Behavior: peripheral accepts exactly one 128-bit block per START; ignores further writes until DONE.
 - Pros: easiest to implement and test.
 - Cons: callers must break every message into 16-byte chunks, manage IV/counter/padding themselves, and issue multiple STARTs — error prone.
2. LEN-based batch processing (recommended minimal improvement)
 - Behavior: CPU writes LEN = number of 16-byte blocks, streams LEN×4 words into DATA_IN, writes START; peripheral processes LEN blocks and raises DONE.
 - Pros: small extra logic, clear semantics, avoids per-block START overhead, good for fixed-size transfers.
 - Cons: still requires caller to submit full blocks (padding for final partial block) or handle partial block separately.
3. Streaming (best for general use; strongly recommended)
 - Behavior: peripheral streaming mode (CTR ideal): CPU sets START (or sets START=streaming), then streams 32-bit words into DATA_IN as they arrive. Peripheral assembles blocks internally and processes them one by one, producing DATA_OUT words as they finish. ABORT or STOP ends stream.

- Pros: supports arbitrary-length messages, partial final blocks handled simply in CTR by XORing only needed keystream bytes (no padding), best UX for DMA-like producers or real-time feeds.
- Cons: a bit more peripheral logic (FIFOs + partial-block handling), but you already planned FIFOs so this is natural.

Handling partial (non-16-byte) final blocks

- CTR mode: simplest — generate keystream for the last block and XOR only the bytes you need; no padding required.
- ECB/CBC modes: you must pad final block (PKCS#7, zero-pad, etc.) or send an explicit length and have peripheral handle stripping — adds complexity.
- Recommendation: support CTR streaming if you want to avoid padding complexity.

Practical tradeoffs & recommended default

- If you have time, implement streaming CTR with input & output FIFOs and a LEN register for convenience — supports both streaming and bounded batches.
- If you're short on time, implement LEN-based batch processing (option 2) first; add streaming later.

Concrete API patterns

A) LEN-based batch (peripheral side)

- CPU:
 - write KEY, IV
 - write LEN = number_of_blocks
 - write N×4 words into DATA_IN (or interleave writes while peripheral consumes)
 - write CTRL.START=1
 - wait for STATUS.DONE or IRQ
 - read N×4 words from DATA_OUT
- Requirements: callers must pad final block (if message length not multiple of 16).

B) Streaming CTR (peripheral side)

- CPU:
 - write KEY, IV (unique per message)
 - write CTRL.START=1 (enter streaming)
 - write plaintext words to DATA_IN as they arrive (poll STATUS.INPUT_FULL or use IRQ)
 - stop by writing CTRL.ABORT=1 or CTRL.START=0; read all DATA_OUT
- Partial final block:
 - CPU can send only the remaining bytes as packed words (e.g., final 1–3 bytes into low bytes of a word). Peripheral must support partial-word writes or software

must pack into 32-bit words and write LEN in bytes. Simpler: require 32-bit writes only and software packs bytes into words.

Example software helpers (pseudocode)

- Chunking for LEN-based batch (splitting a message): for (offset=0; offset<msg_len; offset+=16) { block_len = min(16, msg_len-offset); if (block_len < 16) pad_block(...); // only for ECB/CBC; not needed for CTR write_block_to_data_in(block); } write_reg(PERIPH_BASE + LEN, blocks); write_reg(PERIPH_BASE + CTRL, START | IRQ_EN); wait_done_and_read_outputs();
- Streaming CTR (preferred): write_key_and_iv(); write_reg(CTRL, START | IRQ_EN); // enable streaming for each 32-bit word in packed_plaintext { while (STATUS.INPUT_FULL) wait(); write_reg(DATA_IN, word); } write_reg(CTRL, ABORT) or write_reg(CTRL, START=0); // stop streaming // read outputs until OUTPUT_COUNT == 0

Edge cases and points to design into the peripheral

- How to pack partial bytes:
 - Easiest: require software to pack bytes into 32-bit words (pad the last word with zeros and provide a separate byte-length field so peripheral trims output), or support byte/halfword writes and implement a write assembler in peripheral.
- FIFO depth:
 - Ensure input FIFO can buffer bursts; default 64 words gives reasonable buffering for CPU bursts.
- Counter/IV management:
 - For CTR, peripheral should increment counter per block and store IV; CPU must ensure IV uniqueness per message.
- ABORT semantics:
 - ABORT should allow graceful stop; peripheral should finish current block to produce deterministic output, then stop.
- LEN vs streaming interplay:
 - If LEN is set, peripheral can stop automatically after processing LEN blocks (good for bounded operations); otherwise run until aborted.

Security notes

- For CTR, never reuse (key, IV) pair across different messages. Recommend peripherals expose IV_WRITE and require unique IV per message.
- If peripheral supports LEN, ensure partial-block behavior is documented so software does not leak data.

Final recommendation

- Implement streaming CTR with FIFOs (preferred). It supports arbitrary-length messages, partial final blocks without padding, and fits your iterative one-round-per-clock design.
- If time is short, implement LEN-based batch processing first (easier) and make sure the peripheral API and FIFOs are designed so streaming can be added later without breaking software.

If you'd like, I can now:

- Generate the Verilog AES peripheral (iterative rounds) in streaming CTR mode with input/output FIFOs and partial-block handling, plus cocotb tests and example C driver; or
- Generate a simpler LEN-based batch AES peripheral (faster) that you can extend later.

Which option do you want me to produce?

You said: why would we have a separate start

why would we have a separate start and abort

Copilot said: Short answer - START and ABORT are separate

Short answer

- START and ABORT are separate because they do different jobs: START tells the peripheral to begin or enable work, while ABORT requests it stop or cancel work already in progress. Keeping them separate makes control unambiguous, simpler to reason about, and safer in the face of races, errors, and different operating modes (batch vs streaming).

Why separate bits are useful (concrete reasons)

- Clear intent and symmetry
 - START expresses “begin this operation” (or “enter streaming mode”).
 - ABORT expresses “cancel/stop this operation”.
 - Keeping both makes software/state machines simpler — you explicitly start and explicitly cancel.
- Different semantics / latencies
 - START is usually accepted immediately and transitions peripheral into BUSY state.
 - ABORT may be handled gracefully (finish current block) or immediately, which takes time; separate bit lets firmware issue it without confusing it for a start.
- Cleaner software flow & fewer bugs
 - If you overload START to also perform abort (e.g., toggle START=0 to stop), you must read/modify/ write the same register bit, which can race or be ambiguous when multiple agents/interrupts act.
 - A dedicated ABORT command makes cancellation idempotent and easy to invoke from an ISR.

- Error handling & recovery
 - ABORT can signal a special error/aborted state that START does not. Firmware can poll STATUS.ABORTED or read IRQ to discover abort result.
- Mode differences (batch vs streaming)
 - In batch mode START typically triggers a single bounded run (LEN blocks).
 - In streaming mode START might enable continuous processing until ABORT or STOP — separate ABORT allows controlled shutdown without confusion.

Recommended semantics (practical, unambiguous)

- CTRL.START: a write-1 command to start operation
 - Behavior:
 - If peripheral is IDLE, issuing START sets an internal running flag and (for batch) begins processing LEN blocks or (for streaming) enables continuous processing.
 - Peripheral sets STATUS.BUSY = 1 when it has accepted START.
 - START can be self-clearing (clear as soon as accepted) or sticky (remain set). I recommend START be a command (write-1 to start) and the peripheral clears the START request internally once it transitions to BUSY. This avoids software having to clear the bit later.
 - If START is written while BUSY, peripheral should either ignore the command and optionally set an ERROR bit, or return an IRQ_OVERFLOW/ERROR. Recommended: ignore and set ERROR or return a status so software knows it tried to start while busy.
- CTRL.ABORT: a write-1 command to request cancellation
 - Behavior:
 - ABORT sets an internal abort_request flag immediately; peripheral FSM observes it and performs an abort sequence.
 - Peripheral should acknowledge ABORT by clearing BUSY when done, setting STATUS.ABORTED or STATUS.ERROR (or set a bit in IRQ_STATUS), and optionally asserting IRQ (if IRQ_EN).
 - ABORT should be idempotent: multiple ABORT writes do the same effect as one.
 - ABORT should not implicitly change the key or other persistent config unless explicitly designed to.
- Distinguish STOP vs ABORT (optional)
 - If streaming needs a gentle stop (finish current block(s) and then stop) vs an immediate interrupt-style cancel, you can provide separate STOP (graceful) and ABORT (immediate/zeroize) commands. If you don't need both, make ABORT graceful (finish block then stop) as the default.

Graceful vs immediate abort (recommendation)

- Recommend graceful abort by default:
 - Finish processing the in-flight block so output is consistent.

- Stall/unload only once output FIFO has space for the block if necessary.
 - After finishing, clear BUSY, set STATUS.ABORTED (or ERROR), and assert IRQ if enabled.
- Immediate abort is only needed in special security cases (zeroize keys, stop immediately). If you need zeroize, provide a separate CLEAR_KEYS command or a CONFIG bit with careful semantics.

Implementation notes / FSM sketch

- States: IDLE, KEY_EXPAND, LOAD_BLOCK, ROUND_LOOP (1..10), FINAL_ROUND, UNLOAD_BLOCK, STALL_FOR_OUTPUT, DONE
- Variables: start_req (set on START write), abort_req (set on ABORT write)
- Behavior:
 - On START when in IDLE:
 - Validate preconditions (KEY_READY, enough input words or LEN set). If ok, accept START: start_req -> clear, set busy, move to LOAD_BLOCK or streaming mode.
 - If not ok, set ERROR and ignore start.
 - During processing:
 - On abort_req: set aborted flag; finish current block (or current round if immediate abort requested), then move to DONE/IDLE and set STATUS.ABORTED. Clear abort_req.
 - On DONE:
 - Clear BUSY, set STATUS.DONE, optionally assert IRQ.
- Race handling:
 - If START arrives while BUSY: ignore and set ERROR bit or return error code in STATUS. This avoids double-start races.
 - If ABORT arrives while not busy: treat as no-op but may set STATUS.ABORTED = 0 or set a sticky bit indicating abort was requested but nothing to abort (or simply ignore silently).

Software patterns (examples)

- Batch run (LEN-based):
 - write KEY/IV
 - write data into FIFO / internal RAM
 - write LEN
 - write CTRL.START = 1
 - wait for STATUS.DONE or IRQ
 - read outputs
- Cancel a long streaming transfer:
 - In ISR (or main thread) detect condition → write CTRL.ABORT = 1
 - Wait until STATUS.BUSY == 0 (or wait for IRQ indicating abort complete)
 - Examine STATUS.ABORTED/ERROR and handle cleanup (clear input FIFO, reset counters, etc.)

Why not combine start/abort into one field?

- Ambiguity and race conditions: toggling a single bit to mean both start and stop complicates both firmware and FSM design. It becomes unclear whether a write is a command or a mode change; two separate bit fields are semantically clearer and simpler to implement safely.
- Simpler, safer interrupt/ISR behavior: an ISR can atomically write ABORT without reading-modifying-writing the START bit (which might race with other context).
- Idempotency: separate ABORT can be safe to call multiple times; combining would require toggling semantics that can be fragile.

Small practical suggestions for your AES peripheral

- CTRL layout (write-1 commands):
 - bit0 START (write 1 to start operation); peripheral latches and clears internally when accepted.
 - bit1 ABORT (write 1 to request abort); peripheral latches until handled and then clears.
 - bit2 IRQ_EN
 - bit4 KEY_LOCK (write-1 to set)
- STATUS bits:
 - BUSY, DONE, ABORTED/ERROR, KEY_READY, INPUT_COUNT, OUTPUT_COUNT
- Make ABORT idempotent and graceful by default; provide separate CLEAR_KEYS or RESET for destroying keys.

Would you like a tiny Verilog FSM snippet that shows how to accept START and ABORT (with graceful abort) and how to expose STATUS bits / IRQ? I can generate that and tie it into the FIFO / AES-round loop we discussed.

You said: could I get rid of the separate

could I get rid of the separate start and abort?

Copilot said: Yes — you can get rid of

Yes — you can get rid of separate START and ABORT bits, but you should pick a replacement that preserves the useful semantics those two commands gave you (atomicity, idempotency, clear intent, safe from races). Below are the practical options, their tradeoffs, and my recommendation.

Options

1. Keep them as-is (baseline — recommended)
 - START = begin operation (write-1 command). ABORT = request cancel (write-1 command).

- Pros: very clear semantics, easy to call from ISRs, no read-modify-write races, simple FSM.
 - Cons: two control bits.
2. Replace START + ABORT with a single write-only COMMAND register (recommended if you want fewer control fields)
 - COMMAND (write-only, one 32-bit reg) where software writes an opcode:
 - 0x01 = START
 - 0x02 = STOP (graceful stop / disable streaming)
 - 0x03 = ABORT (cancel now / graceful by policy)
 - (others reserved)
 - Peripheral latches the command, acts on it, and clears the command internally (self-clearing).
 - Status bits (BUSY, DONE, ABORTED, ERROR) indicate outcome.
 - Pros: preserves distinct semantics (start vs abort vs stop) while using a single register and safe write-only semantics (no RMW). Easy to call from ISR. Extensible.
 - Cons: a tiny bit less direct than a dedicated START bit, but semantically clearer than overloading a single bit.
 3. Use START bit only, and use START=0 to stop (NOT recommended)
 - START is a sticky bit: write 1 to start, write 0 to stop.
 - Problems:
 - To write START=0 firmware usually must do a read-modify-write on the control register — races with concurrent events.
 - From an ISR you may not be able to atomically clear START without interfering with other bits.
 - Harder to have an immediate ABORT command; you must define whether clearing START is graceful or immediate.
 - Pros: minimal register footprint.
 - Cons: fragile, less safe, harder to use from interrupts.
 4. Rely on LEN-only batch mode and drop START/ABORT entirely
 - CPU writes LEN and enough DATA_IN; peripheral processes LEN blocks and stops automatically.
 - Pros: no explicit start/abort needed, simple.
 - Cons: doesn't support streaming or interactive cancellation; aborting requires reset. Not suitable if you need streaming/long-running ops.
 5. Use a single-write START/STOP bit convention (write-1-to-start, write-1-to-stop)
 - Two command bits in the same register but each is write-1-only and self-clearing.
 - This is effectively the command register idea but with two bit fields instead of opcode values.
 - Pros: avoids RMW and keeps simple read-friendly status.
 - Cons: slightly less extensible than an opcode field.

Important implementation details whichever you choose

- Make control writes write-only commands (self-clearing) so software doesn't need RMW and ISRs can issue commands safely.

- Provide clear STATUS bits (BUSY, DONE, ABORTED/ERROR, KEY_READY) so software can observe effects.
- ABORT semantics: document whether it's graceful (finish current block) or immediate, and how long it may take to take effect.
- If you keep streaming mode, you should keep a way to stop/abort from an ISR — don't force a full reset.

My recommendation

- If you want to remove the separate START and ABORT bits but keep the same functionality, use a single write-only COMMAND register with opcodes for START, STOP and ABORT. It's compact, safe, and extensible. Example register usage:
 - write COMMAND = 0x01 // start
 - write COMMAND = 0x03 // abort
 - read STATUS to learn if BUSY cleared / ABORTED set
- If you don't need streaming and only do fixed batches (LEN-based), then you can safely drop ABORT and just use LEN + a single START command — but you lose run-time cancellation.

Would you like a tiny example API and a short Verilog FSM snippet showing a write-only COMMAND register implementation (self-clearing) and how STATUS bits reflect results?

You said: we are ujst going to have a

we are ujst going to have a start bit, when it is 0 the system resets

Copilot said: Okay — that's a simple control model

Okay — that's a simple control model and we can make it work. Here's what it means, the tradeoffs, recommended safe semantics, implementation notes (including how to wire it so it behaves cleanly), and a suggested software usage pattern. At the end I can generate the Verilog FSM/register logic that implements this if you want.

Summary of your choice

- Single control bit: CTRL.START at 0x00 bit0.
- Semantics you proposed: START == 1 => peripheral operates; START == 0 => peripheral resets (stops).
- No separate ABORT; stopping is done by writing START = 0.

Implications and tradeoffs

- Simplicity: very simple to reason about — peripheral is active only while START==1.
- Reset-on-stop behavior: resetting on START==0 can be implemented as either a "soft reset" (clear internal state/FIFOs but keep key) or a "hard reset" (also zeroize key

material). You must choose which — both are reasonable but have different security/UX consequences.

- Race and atomicity: clearing START by writing the control register may require read-modify-write if there are other control bits in the same register; that can race with concurrent code/interrupts. To avoid that, either:
 - Put START in a register by itself (only bit in its register), or
 - Make START write-only with defined semantics (write 1 to start, write 1 to stop) or use a dedicated COMMAND register. Since you said you want only a START bit and START==0 resets, best practice is to keep START as a single-bit register (no other persistent bits in that same word).
- Immediate stop vs graceful stop: reset-on-zero implies immediate stop. That is simpler but can leave partially processed blocks / inconsistent outputs if you don't also handle finishing the current block before reset. If you want consistent output, implement START==0 as a soft reset that finishes current block before clearing; but that means it's not instantly resetting. Decide which you prefer.

Recommended semantics (safe, explicit)

- Use START as a level-controlled enable with the following behavior:
 - START == 0: peripheral is held in reset/idle. Internal state is cleared to known defaults; FIFOs are flushed. Peripheral does not accept DATA_IN writes for processing (you may still allow KEY writes while START==0).
 - START == 1: peripheral leaves reset and is ready to accept KEY, IV, and DATA_IN (or if KEY and IV already present it can start processing immediately depending on LEN/streaming semantics).
- Make the reset synchronous and deterministic:
 - When START transitions 0->1: initialize FSM, set STATUS.BUSY=0, KEY_READY cleared until key expansion completes, etc.
 - When START transitions 1->0: immediately stop processing and move to reset state; clear BUSY and optionally zeroize keys depending on chosen policy.
- Key zeroize policy (pick one):
 - Hard reset: START==0 zeroizes key material and expanded round keys. Good for security, but inconvenient if you want to pause/resume without reloading key.
 - Soft reset (recommended if you want convenience): START==0 clears FIFOs and aborts operation but preserves the loaded key and the expanded round keys. This allows the software to stop/start without rewriting the key. If you need zeroize, provide a separate explicit CLEAR_KEYS register or make START==0 + CONFIG.ZEROIZE set cause zeroization.
- Make KEY_LOCK independent of START or accept that START==0 resets KEY_LOCK (choose consistent behavior). If KEY_LOCK is important (prevent accidental overwrite), keep it in a separate register that is not cleared by stopping unless you want it cleared.

Concrete register and behavior recommendation given your choice

- Put START as the only bit in CTRL (so software can write 0 or 1 without RMW concerns), or at minimum make CTRL a dedicated 32-bit reg where bit0 is START and other persistent controls are in separate registers.
- Example behavior:
 - On reset (including START==0): clear internal FSM, head/tail pointers of FIFOs, set STATUS.BUSY=0, STATUS.DONE=0, optionally clear STATUS.ERROR. Optionally preserve KEY/round-keys (soft reset) or zeroize them (hard reset).
 - On START==1: peripheral ready to accept KEY, IV, DATA_IN; software sequence: write KEY, IV, write LEN/enable streaming if needed, then write START=1.
 - If START is deasserted mid-block, the peripheral will immediately abort and clear internal state (or if you prefer, finish the block then reset — pick one).
- Recommended: START==0 performs a soft reset that clears FIFOs & FSM, but preserves the key. If you want higher security, add an explicit CLEAR_KEY command that zeroizes keys.

How this affects software usage

- Initialization / start:
 - Ensure START==0 (peripheral idle)
 - Write KEY0..KEY3 and IV0..IV3 as needed
 - Wait KEY_READY or configured delay for key expansion
 - Fill DATA_IN as desired or set LEN and then write START=1
- Stop/reset:
 - To stop processing, write START = 0. Peripheral will go to reset state (soft or hard depending on policy).
 - If you preserve key on reset, you can later write START=1 again without rewriting key.
 - If you zeroize key on START==0, you must rewrite key to restart.

Implementation notes (hardware)

- Implement START as a synchronous enable/reset:
 - On each clock, sample start_reg. If start_reg == 0 then assert internal reset signals (clear pointers, clear FSM state, optionally zero key arrays).
 - Avoid asynchronous resets for much of the logic to keep timing simple; use synchronous clears keyed to the sampled start value.
- Peripheral registers while START==0:
 - Allow writes to KEY0..KEY3 and IV regs while START==0. These get latched and key expansion can be performed once START==1 or you can expand immediately on KEY3 write even with START==0 (makes starting faster).
 - Block DATA_IN writes from being accepted into the FIFO while START==0, or accept them into FIFO but do not process until START==1 — either is acceptable, but blocking DATA_IN until START==1 avoids accidental queued data.

- Handling in-flight block when START goes 1->0:
 - Immediate reset: simply clear state and pointers. Any partially processed block result is discarded. CPU should not expect output for that block.
 - Graceful finish: continue until current block completes, then enter reset state. This requires storing an abort-request flag rather than immediate reset. It's more deterministic for consumers who expect fully-formed outputs.
- Provide STATUS bits so software knows peripheral state: BUSY, DONE, KEY_READY, FIFO counts, ERROR.

Pitfalls & gotchas

- Writing START=0 via RMW if CTRL has other bits will be fragile; keep START separate or have other control bits in different registers.
- If START==0 clears the key, you will force software to rewrite the key to restart — which may be acceptable or not depending on usage.
- If you allow KEY writes only when START==0, ensure code sequences adhere to that (write key then set START=1).
- If memory-mapped writes to DATA_IN are allowed while START==0 and are buffered, make sure input FIFO behavior is documented.

Example small Verilog control sketch (conceptual)

- At a high level:
 - reg start_reg;
 - always @(posedge clk) start_reg <= ctrl_write ? ctrl_in[0] : start_reg;
 - always @(posedge clk) if (!start_reg) begin reset_internal_state(); end else begin normal_operation(); end
- "reset_internal_state()" clears FIFO pointers, counters, FSM state, IRQ flags; optionally clears key material if you selected hard reset.

Recommendation (my pick)

- Use START as the only persistent control bit in CTRL (no ABORT). Implement START==0 as a synchronous soft reset that clears FIFOs and aborts processing but preserves the key/round keys. Add a separate explicit CLEAR_KEYS register for zeroizing keys when you need it. This gives safe, restartable behavior without needing ABORT while still enabling secure zeroization when required.

Do you want me to:

3. Generate the Verilog control logic + FIFO reset behavior where START==0 performs a synchronous soft reset (preserves key; clears FIFOs), or
4. Generate the Verilog where START==0 performs a hard reset (zeroizes key + clears everything), or
5. Provide both variants and a short C example showing correct software sequences for each?

Tell me which reset policy you want (soft preserve-key or hard zeroize-key) and whether START is the only bit at the CTRL register (I'll assume yes). I'll then produce the Verilog module skeleton that implements it and tie it into the AES FSM / FIFOs.

You said: how large are the address bits

how large are the address bits

Copilot said: Short answer - The TinyQV peripheral address bus

Short answer

- The TinyQV peripheral address bus is 28 bits wide: `qv_data_addr[27:0]`. The top 3 bits (`qv_data_addr[27:25]`) are used to decide memory vs peripheral region, so peripherals are addressed within that 28-bit space. Registers are typically 32-bit (word) aligned, so the low two address bits are normally zero for word accesses.

Details and practical notes

- Address width: 28 bits (addresses $0..2^{28}-1$). You saw this earlier where code checks `qv_data_addr[27:25]` to distinguish mem vs peripheral.
- Peripheral base example: `0x0800_0000` (hex) = `28'h8000000`. That value fits within the 28-bit bus.
- Offsets: to index registers inside your peripheral you compute an offset = `qv_data_addr - PERIPH_BASE` (or simply use `addr[? : 0]` if you decode by high bits). For 32-bit registers you usually use `offset[31:2]` (in this case `offset[27:2]`) or just `offset[5:2]` to select small register windows.
- Alignment: since CPU writes are normally 32-bit, expect `offset[1:0] == 2'b00` for normal register accesses. If you support byte/halfword accesses, use the `data_write_n` size signals to pack/assemble.
- Decoding example (Verilog-ish):
 - `wire [27:0] addr = qv_data_addr;`
 - `wire is_periph = (addr[27:25] != 3'b000);`
 - `localparam PERIPH_BASE = 28'h8000000; // 0x0800_0000`
 - `wire [27:0] periph_offset = addr - PERIPH_BASE; // offset within your peripheral`
 - `// select 32-bit aligned register index: wire [5:0] reg_index = periph_offset[7:2]; // example: supports 64 registers (0x00..0xFC)`
- Register space sizing: with a 28-bit address and 32-bit alignment you have up to $2^{(28-2)} = 2^{26} = 67,108,864$ possible 32-bit-register words in the whole address space; your single peripheral will use only a small subrange (pick offsets `0x00..0x??`).
- Check top-level signals: if you want to be 100% sure for this repo, look at the top-level module (`iceFUN.v` / `tinyqv.v`) to confirm `qv_data_addr` width and the exact naming; the earlier messages indicate `qv_data_addr[27:25]` is used, so 28 bits is correct.

If you want, I can:

- Produce a tiny Verilog decode+register-slice snippet for PERIPH_BASE = 28'h8000000 that maps the register offsets you defined (CTRL, STATUS, KEY, DATA_IN/OUT, etc.), or
- Show how to compute reg_index and handle 32/16/8-bit write sizes with data_write_n. Which would you prefer?

You said: how does the peripheral know when to

how does the peripheral know when to output the answer onto the bus?

Copilot said: Short answer - The peripheral drives a read

Short answer

- The peripheral drives a read result onto the CPU bus when the CPU issues a read to that peripheral register and the peripheral has valid data ready. Concretely: on a matched read address the peripheral must present the 32-bit word on `data_in` and assert `data_ready` for the cycle the read is serviced. The usual, robust implementation is to keep the FIFO's next-word pre-fetched into a register so the peripheral can respond in one cycle when the CPU reads DATA_OUT.

How it actually works (step-by-step)

1. CPU issues a read transaction to an address (e.g., PERIPH_BASE + 0x34 for DATA_OUT).
 - On TinyQV this is seen as a read when `data_read_n != 2'b11` and the decoded address matches your register offset.
2. Peripheral checks whether it has valid data for that register:
 - For STATUS/CTRL-like registers the data is usually always available, so the peripheral can immediately drive `data_in` and assert `data_ready`.
 - For the output FIFO (DATA_OUT) it must only respond if the FIFO is not empty (has at least one word). If empty it should not assert `data_ready` (or else return an error/zero as per your policy).
3. Presenting the data and asserting the handshake:
 - In the same cycle the peripheral must drive `data_in` (the 32-bit value) and assert `data_ready` to indicate the read completed.
 - The CPU/mem_ctrl will see `data_ready` and sample the returned word.
4. Advance the FIFO head:
 - When the read is accepted, the FIFO head pointer should increment and the FIFO count be decremented (take care to handle simultaneous push and pop in the same clock).

Timing caveat: RAM read latency

- If the FIFO is implemented with synchronous BRAM (one-cycle read latency), you cannot read `mem[head]` and present it in the same cycle you decide to pop — the BRAM output will only be available the next clock. Two common solutions:
 1. Prefetch model (recommended for single-cycle response): always keep the word at `mem[head]` registered into `out_reg` (on every cycle sample `mem[head]` into `out_reg` so it's already available when a read arrives). On a pop you drive `out_reg` on `data_in` and assert `data_ready`. Also kick off a BRAM read for the next head to refill `out_reg`.
 2. Two-cycle reply: when a read arrives and FIFO not empty, accept the read request but assert `data_ready` only in the next cycle when BRAM output is available. This stalls the CPU read for one extra cycle; it works but is less convenient for software that expects one-cycle reads.

Correct handling of simultaneous push & pop

- If producer (AES core) writes and CPU reads in the same cycle, both push and pop can happen simultaneously. Update tail and head and adjust count by $(+push - pop)$. The presented `out_reg` must reflect the correct pre-read value for the pop case.

A simple Verilog-style snippet (prefetch single-cycle reply)

- This snippet demonstrates: prefetch `out_reg`, detect a CPU read to `DATA_OUT`, assert `data_ready` and advance head. It assumes 32-bit-only reads and a peripheral address `DATA_OUT_ADDR`.

Verilog

```
// Parameters / signals assumed:
// mem[DEPTH-1:0] : synchronous BRAM or reg array
// head, tail, count : FIFO pointers
// out_reg : registered next-out word (valid when count > 0)
// cpu_addr, cpu_read_n : bus inputs
// DATA_OUT_ADDR : decoded address offset for DATA_OUT
// data_in (output to mem_ctrl), data_ready (output to mem_ctrl)

wire cpu_read = (cpu_addr == DATA_OUT_ADDR) && (cpu_read_n != 2'b11);
wire fifo_not_empty = (count > 0);
reg data_ready_reg;
reg [31:0] data_in_reg;

always @(posedge clk) begin
    if (!rstn) begin
        head <= 0; tail <= 0; count <= 0;
```

```

    out_reg <= 32'd0;
    data_ready_reg <= 1'b0;
end else begin
    // Prefetch: keep out_reg loaded with mem[head] whenever FIFO not empty
    if (fifo_not_empty) begin
        // For BRAM: assume synchronous read, so you initiated read earlier.
        out_reg <= bram_read_data; // bram_read_data is BRAM output registered last cycle
    end

    // detect CPU read to DATA_OUT and FIFO has data -> service immediately
    if (cpu_read && fifo_not_empty) begin
        data_in_reg <= out_reg;    // drive the bus word this cycle
        data_ready_reg <= 1'b1;    // one-cycle ack to mem_ctrl
        head <= head + 1;
        count <= count - 1;
        // start BRAM read for new head to refill out_reg next cycle
        bram_read_address <= head + 1;
    end else begin
        data_ready_reg <= 1'b0;
    end

    // Handle producer writes (AES core) and other logic...
    // push = producer_valid && (count < DEPTH)
    // if (push) begin mem[tail] <= producer_data; tail <= tail + 1; count <= count + 1; end

end
end

assign data_in = data_in_reg;
assign data_ready = data_ready_reg;

```

Practical software patterns to avoid surprises

- Software should normally poll `STATUS.OUTPUT_COUNT` (or `OUTPUT_EMPTY` bit) and only issue reads when output is available. That avoids relying on `data_ready` timing and avoids triggering underflow errors.
- Provide IRQ thresholds so software can be notified when enough output words are available to read in a burst.
- If you want the simplest peripheral, require 32-bit reads only, and always prefetch BRAM output to be able to respond in a single cycle.

Summary checklist for correct peripheral read behavior

- Decode CPU read to the correct register address.

- Only assert `data_ready` when you actually have the correct 32-bit value stable on `data_in`.
- For FIFO DATA_OUT, present the head word and assert `data_ready` when FIFO is not empty; then increment head and decrement count.
- If using synchronous RAM, prefetch the RAM output into a registered `out_reg` so the word is available immediately when a read arrives (single-cycle reply).
- Expose STATUS/COUNT bits and IRQs so software rarely needs to poke reads when output is absent.

If you want, I can:

- Provide a complete, parameterized Verilog FIFO module with registered output and single-cycle pop behavior suitable for your AES peripheral, or
- Produce the exact read-handshake snippet integrated into the AES peripheral register decode logic so you can paste it into your top-level. Which would you prefer?

You said: we are no longer using a fifo

we are no longer using a fifo

Copilot said: If you remove the FIFO, the peripheral

If you remove the FIFO, the peripheral still needs a place to hold completed ciphertext/plaintext so the CPU can read it. In practice that means a small result buffer (registers) plus well-defined handshaking: the AES core writes the result registers when a block completes, the peripheral sets status/IRQ, and the CPU reads those registers. Below I explain the common options, recommend one, and give the precise register/behavior pattern and a Verilog-style sketch you can drop into your peripheral.

Key points up front

- You must buffer completed blocks somewhere (result registers). A block is 128 bits = 4×32 -bit words on your bus.
- The peripheral must signal availability (STATUS.DONE / RESULT_VALID) and optionally IRQ so CPU knows when to read.
- CPU reads DATA_OUT (or RESULTn) and that read should decrement an internal read counter; when all 4 words are read the peripheral clears DONE/RESULT_VALID so a new block can be produced.
- If CPU doesn't read fast enough, the peripheral should stall producing the next block (backpressure) or raise an error/overflow — do not silently overwrite previous result.

Recommended design (no FIFO)

- Result buffer: 4×32 -bit result registers `result[0..3]`.
- State flags:

- result_valid (1 if a complete 128-bit result is available and unread)
 - result_index (0..3) index of next word to return on DATA_OUT read
 - busy (AES processing in progress)
- Behavior on AES completion:
 - AES core finishes processing a block.
 - If result_valid == 0 (previous result consumed): write result[0..3] in registers, set result_valid = 1, result_index = 0, set STATUS.DONE = 1 (and BUSY = 0), assert IRQ if enabled.
 - If result_valid == 1 (previous result not yet read): either (A) stall AES core before committing this new result (preferred), or (B) set ERROR/OVERFLOW and drop/overwrite (not recommended). Stall means AES core must not start the next block until results are consumed.
- Behavior on CPU read DATA_OUT:
 - If result_valid == 1 and result_index < 4: drive data_in = result[result_index], assert data_ready for the read cycle, increment result_index.
 - If result_index becomes 4 after increment, clear result_valid and STATUS.DONE (and optionally clear IRQ).
 - If DATA_OUT read arrives when result_valid == 0: do not assert data_ready (the read stalls), or optionally set an ERROR/UNDERFLOW bit. Recommended: do not assert data_ready and let software check STATUS before reading.
- How CPU should use it:
 - Poll STATUS.DONE or wait for IRQ, then read DATA_OUT 4 times (or read RESULT0..3 if you expose them).
 - After reading all 4 words, you can start the next block by writing START=1 (or the peripheral will accept next block automatically if you support streaming and the result was cleared).
- Optional extra registers:
 - RESULT_COUNT or OUTPUT_COUNT in STATUS (0..4) so CPU can avoid reading blindly.
 - A single-word DATA_OUT register that auto-increments index on read (simpler to use).
 - RESULT0..RESULT3 fixed registers (allows random access reads but then you must decide if reads clear VALID or not — simpler to leave them readable always and clear VALID only after reads to DATA_OUT).

Timing considerations

- Synchronous RAM/read timing: prefer to present result words from registers (not BRAM) so you can immediately drive data_in and assert data_ready in the same cycle the CPU reads (single-cycle response).
- Race case (AES finishes exactly in same cycle CPU issues read): to avoid tricky races, ensure AES writes result registers and sets result_valid in one clock edge; then the CPU will read in a later cycle. If your platform can do single-cycle read response, you can also design it so CPU read concurrently with the write is resolved deterministically, but simpler and safer is to require CPU to poll STATUS/IRQ then read.

Suggested register map tweaks (no FIFO)

- Keep earlier registers but adjust STATUS and DATA_OUT semantics:
 - 0x00 CTRL: START (level/command as you decided)
 - 0x04 STATUS:
 - bit0 BUSY
 - bit1 DONE (alias result_valid)
 - bit2 INPUT_FULL (not used if no input FIFO; use as "input busy")
 - bit3 OUTPUT_EMPTY (== !result_valid)
 - bit4 ERROR
 - bits15:8 INPUT_COUNT (optional; 0 if no input FIFO)
 - bits23:16 OUTPUT_COUNT (0..4: number of unread 32-bit words available)
 - 0x30 DATA_IN (W): CPU writes 4 words to trigger AES (or write them before START)
 - 0x34 DATA_OUT (R): returns next 32-bit word from result buffer (auto-increment result_index)
 - 0x38 IRQ_STATUS/CLEAR etc.

Practical policy for input (since no FIFO)

- Option A (single-block only): peripheral accepts exactly four DATA_IN writes (in order) to fill input buffer; when START=1 you process that single block and produce result. Software must wait for DONE, read result, then write next block.
- Option B (LEN/batch without FIFO): require CPU to write input block N times into the block register and START each time; or write block data into memory and have CPU initiate multiple STARTs. Simpler: implement single-block processing first.
- Option C (streaming without FIFO but still streaming): accept DATA_IN writes and if core is busy, return an error or block the write by not asserting data_ready. This is awkward without FIFO because CPU writes would stall until core accepts data; you must design for that.

Example Verilog-ish sketch for read side (single result buffer, auto-increment on read)

```
Verilog
// registers
reg [31:0] result_reg [0:3];
reg      result_valid;
reg [1:0] result_index; // 0..3
reg      busy;
reg      irq; // or use IRQ_STATUS

// AES completion: called when AES core finishes block
task aes_block_done(input [127:0] block_out);
begin
```

```

if (!result_valid) begin
    {result_reg[3], result_reg[2], result_reg[1], result_reg[0]} <= block_out; // choose word order
    result_valid <= 1'b1;
    result_index <= 2'd0;
    busy <= 1'b0;
    irq <= (ctrl_irq_en) ? 1'b1 : 1'b0;
end else begin
    // previous result not consumed: stall/start policy
    // recommended: assert error and do not overwrite
    error <= 1'b1;
end
end
endtask

```

```

// CPU read handling (DATA_OUT_ADDR decoder sets cpu_read)
always @(posedge clk) begin
    if (!rstn) begin
        data_ready_reg <= 1'b0;
        data_in_reg <= 32'd0;
    end else begin
        if (cpu_read && addr == DATA_OUT_ADDR) begin
            if (result_valid) begin
                data_in_reg <= result_reg[result_index];
                data_ready_reg <= 1'b1;
                result_index <= result_index + 1;
                if (result_index == 2'd3) begin
                    result_valid <= 1'b0;
                    irq <= 1'b0;
                end
            end else begin
                // no data available: do not assert data_ready (read stalls),
                // or set an underflow error bit if you prefer
                data_ready_reg <= 1'b0;
                // optional: underflow <= 1'b1;
            end
        end else begin
            data_ready_reg <= 1'b0;
        end
    end
end
end

```

Guidance & tradeoffs summary

- Advantage of removing FIFO: simpler logic, smaller area. Good for single-block or low-throughput use.
- Disadvantage: no decoupling between CPU and AES core. CPU must not send a new block until it has read the prior result (or core must stall and you must define policy).
- If you want batch/streaming in the future, adding even a tiny FIFO (depth 1 block or a few words) will greatly improve usability. But the register-based approach is perfectly fine for simple single-block or one-at-a-time workflows.
- Always document read/write order and require software to check STATUS.DONE/OUTPUT_COUNT before reading.

Would you like me to:

- Generate a complete Verilog peripheral implementing the no-FIFO variant (single-block input buffer + 4-word result registers + STATUS/IRQ logic + START reset semantics), or
- Produce a small testbench or cocotb test that shows the CPU write → START → AES completes → read DATA_OUT ×4 flow?

Which do you want me to produce next?

You said: how does the CPU tell the AES

how does the CPU tell the AES it is requesting / doing a read?

Copilot said: Short answer - The CPU requests a peripheral

Short answer

- The CPU requests a peripheral read by issuing a normal memory read to the peripheral's address. On the bus-side this appears as the mem/mem_ctrl signals: peripheral address on qv_data_addr and a non-idle read code on data_read_n (not 2'b11). The peripheral detects that read (address match + read code) and must drive the return data on data_in and assert data_ready when it wants the mem_ctrl to complete the read.

More detail — signals & handshake

- Signals you'll typically see from the mem_ctrl/CPU side:
 1. qv_data_addr[27:0] — address the CPU is accessing.
 2. data_read_n[1:0] — read-size encoded: 2'b11 = no access, 2'b10 = 32-bit, 2'b01 = 16-bit, 2'b00 = 8-bit. (So any value != 2'b11 is a read.)
 3. data_write_n[1:0] — similarly encodes writes (peripheral should ignore reads when write is active).
- Peripheral responsibilities:
 1. Decode is_periph (e.g., qv_data_addr[27:25] != 3'b000) and the register offset: offset = qv_data_addr - PERIPH_BASE.

2. Detect a read to a specific register: `cpu_read = is_periph && (data_read_n != 2'b11) && (offset == DATA_OUT_OFFSET)`.
3. If you have valid data ready for that register, drive the 32-bit value onto `data_in` and assert `data_ready` for the cycle you want the read to complete.
4. If you cannot provide data immediately (e.g., BRAM read latency), you can delay asserting `data_ready` until the next cycle when `data_in` is valid. The `mem_ctrl` will complete the read only when it sees `data_ready` asserted.

Timing options

- Single-cycle reply (preferred when possible)
 - Preload or prefetch the value into a register so when `cpu_read` is true you can immediately drive `data_in` and assert `data_ready` in the same cycle. This gives the CPU a one-cycle read completion.
- Multi-cycle reply (safe for synchronous BRAM)
 - If you need to fetch from synchronous BRAM, detect `cpu_read`, schedule the BRAM read, and assert `data_ready` + drive `data_in` on the next cycle once BRAM output is valid. This makes the CPU read take one extra cycle.
- Stalling vs signaling error when no data
 - If `DATA_OUT` is empty (no result ready), the peripheral should NOT assert `data_ready` — the read will simply be held until you do assert `data_ready`. Alternatively, you can return a defined value and set an ERROR/UNDERFLOW bit, but stalling is cleaner if software is expected to poll STATUS first.

Example read-detection pseudocode (verilog-style)

- Assumes `PERIPH_BASE`, `DATA_OUT_OFFSET`, and 32-bit reads only:

```
wire is_periph = (qv_data_addr[27:25] != 3'b000); wire cpu_read = is_periph && (data_read_n != 2'b11); wire read_dataout = cpu_read && (periph_offset == DATA_OUT_OFFSET);
```

```
// Single-cycle reply (out_reg contains next word already) always @(posedge clk) begin if (read_dataout && out_valid) begin data_in_reg <= out_reg; // value to send back data_ready_reg <= 1'b1; // mem_ctrl will accept the read head <= head + 1; // advance result index update_out_valid_and_reg(); // prepare next out_reg if any end else begin data_ready_reg <= 1'b0; end end
```

```
assign data_in = data_in_reg; assign data_ready = data_ready_reg;
```

What the CPU does to initiate the read

- At software level: issue a load from the peripheral address (e.g., read from pointer `PERIPH_BASE + 0x34`). The CPU/`mem_ctrl` places the address on `qv_data_addr` and sets `data_read_n` accordingly. The peripheral must respond as above.

Best practices

- Software should poll STATUS (e.g., STATUS.DONE or OUTPUT_COUNT) before reading DATA_OUT to avoid spinning the bus waiting for data.
- If you expect immediate reads, prefetch the data into an out register so you can respond in one cycle.
- Decide and document the policy for reads when result is not ready (stall vs error). Stalling plus STATUS checks is simplest.

If you want, I can:

- Provide a ready-made Verilog register decode + read-handshake snippet for your DATA_OUT (no-FIFO result buffer) that handles single-cycle response by prefetching result words, or
- Show a two-cycle-safe variant for synchronous block RAM reads. Which do you prefer?

You said: will this work for our address map: 0x00:

will this work for our address map: 0x00: control reg - bit 0: start on 1, reset on 0 - bit 1: key lock
 0x01: status reg - bit 0: 1 on done - bit 1: key ready 0x02: input key 0x06: input data
 0x0A: output data

Copilot said: Short answer: Yes — but only if

Short answer: Yes — but only if you treat those offsets as 32-bit register indices (not byte addresses) and expand the map to hold the full 128-bit key and full 128-bit input/output blocks. Right now your map is missing alignment/word-layout details and the additional registers needed to hold 4×32-bit key words and 4×32-bit data words per block. Below I explain what to change and why, and give a corrected, recommended map and the expected software semantics.

Key points / assumptions

- The CPU and mem_ctrl use 32-bit word accesses. Registers must be 32-bit aligned. In this reply I assume your numbers (0x00, 0x01, 0x02, ...) are register indices (word indices). The byte address = PERIPH_BASE + index*4.
- A 128-bit AES key and a 128-bit block require four 32-bit words each. With no FIFO you must buffer the whole 128-bit input and 128-bit output in registers.
- The peripheral needs status flags for readiness (key expanded), done/result valid, and ideally BUSY and ERROR for robust software handling.

Recommended byte-aligned register map (PERIPH_BASE + offset in bytes)

- 0x00 (0x00): CTRL (RW)
 - bit0: START (1 = operate / enabled; 0 = reset/idle)
 - bit1: KEY_LOCK (write-1 to lock; ignored on reads)
 - (other bits reserved)
- 0x04 (0x01): STATUS (R)

- bit0: DONE / RESULT_VALID (1 when a complete 128-bit result is ready)
- bit1: KEY_READY (1 when key expansion finished)
- bit2: BUSY (optional: 1 while AES core processing)
- bit3: ERROR (overflow/underflow/other)
- bits23:16: OUTPUT_COUNT (0..4 words available)
- bits15:8: INPUT_COUNT (if you want to show how many input words have been written)
- 0x08 (0x02): KEY0 (W only) — first 32 bits of 128-bit key
- 0x0C (0x03): KEY1 (W only)
- 0x10 (0x04): KEY2 (W only)
- 0x14 (0x05): KEY3 (W only) — writing this last word triggers key expansion; reads return 0
- 0x18 (0x06): DATA_IN_0 (W) — input block word 0
- 0x1C (0x07): DATA_IN_1 (W)
- 0x20 (0x08): DATA_IN_2 (W)
- 0x24 (0x09): DATA_IN_3 (W) — when all four have been written, the input buffer is complete
- 0x28 (0x0A): DATA_OUT_0 (R) — read words sequentially
- 0x2C (0x0B): DATA_OUT_1 (R)
- 0x30 (0x0C): DATA_OUT_2 (R)
- 0x34 (0x0D): DATA_OUT_3 (R)
- (optionally add IRQ_STATUS at 0x38, CONFIG at 0x3C, etc.)

Why this change is needed

- Your original map listed only one KEY register and one DATA_IN/DATA_OUT offset. AES-128 requires four 32-bit words for key and four for input and output. The peripheral must provide these to the CPU.
- Registers must be word-aligned; 0x06 and 0x0A as byte offsets are not 32-bit aligned — if you meant register indexes they map to byte addresses 0x18 and 0x28 respectively (see map above).
- STATUS should expose KEY_READY and DONE so software knows when to read outputs and when a key expansion has finished. BUSY/ERROR are useful for robust drivers.

Typical usage sequence (single-block, no FIFO)

1. Ensure CTRL.START == 0 (peripheral idle / reset state).
2. Write KEY0..KEY3 at 0x08..0x14 (if KEY_LOCK set, these writes are ignored).
3. Wait for STATUS.KEY_READY == 1 (key expansion done).
4. Optionally write IV (if CTR) or other config.
5. Write DATA_IN_0..DATA_IN_3 at 0x18..0x24.
6. Write CTRL.START = 1 (or set START earlier depending on your design).
7. Wait for STATUS.DONE (or for STATUS.BUSY to clear); peripheral writes DATA_OUT_0..3 internally.

8. Read DATA_OUT_0..3 at 0x28..0x34.
9. After reading word 3, STATUS.DONE/RESULT_VALID clears and the peripheral may accept the next block.

Behavioral notes / recommendations

- If a new block completes while the previous result has not been read: either stall the AES core (preferred) or set ERROR and drop/overwrite previous result (not recommended).
- Make KEY_LOCK a set-only bit (write-1 to lock) and clear only on reset (or via a special CLEAR_KEYS command) to avoid accidental overwrites.
- Prefer START as a single control bit in its own register (as you specified) so clearing it is atomic and doesn't require read-modify-write.
- For DATA_OUT reads: present the next result word when the CPU issues a read; drive data_in and assert data_ready for that cycle. If no result present, do not assert data_ready (read stalls) or set an underflow ERROR bit.
- If you want streaming or batch later, you can add LEN and a small FIFO; with the no-FIFO design the CPU must follow the single-block protocol.

If you intended the original offsets as word indices (not bytes)

- Your original 0x02 -> byte 0x08, 0x06 -> 0x18, 0x0A -> 0x28. The corrected map above corresponds exactly to that interpretation, but expands KEY and DATA to 4 words each.

Would you like me to:

- Generate the Verilog register decode + skeleton logic for this map (CTRL, STATUS, KEY words, DATA_IN/OUT words, key expansion trigger, result buffer behavior) so you can drop it into the top-level?
- Or produce a small C driver example that demonstrates the write-key / write-data / start / wait-for-done / read-output flow for this concrete map?

Pick which and I'll produce the code.

You said: will this work for our address map: 0x00:

will this work for our address map: 0x00: control reg - bit 0: start on 1, reset on 0 - bit 1: key lock
0x01: status reg - bit 0: 1 on done - bit 1: key ready 0x02: input key 0x12: input data 0x22:output data

Copilot said: Yes — that layout can work, but

Yes — that layout can work, but only if you formalize a few details. Two safe, common options are available; pick one and I'll generate the corresponding glue/FSM. Summary first, then details and recommendations.

Quick summary

- Option A (recommended): treat 0x02 / 0x12 / 0x22 as the base word indices for 4-word fields and expose KEY0..KEY3 at 0x02..0x05, DATA_IN0..3 at 0x12..0x15, DATA_OUT0..3 at 0x22..0x25. This is simple and explicit.
- Option B (compact): keep single addresses (0x02, 0x12, 0x22) and have the peripheral auto-increment an internal index on each write/read so four writes/read cycles fill/empty the 128-bit key/input/output. This is workable but needs explicit rules (ordering, overflow behavior, interaction with START/KEY_LOCK).
- Whichever you pick, remember CPU accesses are 32-bit words — byte address = $\text{PERIPH_BASE} + \text{reg_index} * 4$. So reg index 0x12 maps to byte offset 0x48.

Important points & caveats

1. Word vs byte offsets
 - Your numbers look like register indices (0x00..). The bus uses byte addresses; the peripheral will see the address as $\text{PERIPH_BASE} + \text{index} * 4$. For example:
 - reg 0x02 → byte addr $\text{PERIPH_BASE} + 0x08$
 - reg 0x12 → byte addr $\text{PERIPH_BASE} + 0x48$
 - reg 0x22 → byte addr $\text{PERIPH_BASE} + 0x88$
 - Ensure the top-level mapper/decoder in the system uses the same convention.
2. A 128-bit key / 128-bit block uses four 32-bit words
 - AES-128 key is 4×32 bits; a block is 4×32 bits.
 - You must provide storage for all 4 words. Either by explicit registers KEY0..KEY3/DATA_IN0..3/DATA_OUT0..3, or by an internal write-assemble mechanism that requires 4 writes to a single register address.
3. Recommended explicit map (safe, minimal ambiguity)
 - CTRL (0x00)
 - bit0 START (1=run, 0=reset)
 - bit1 KEY_LOCK (write-1-to-set)
 - STATUS (0x01)
 - bit0 DONE / RESULT_VALID
 - bit1 KEY_READY
 - bit2 BUSY (optional)
 - bit3 ERROR (optional)
 - KEY0 (0x02)
 - KEY1 (0x03)
 - KEY2 (0x04)
 - KEY3 (0x05) // writing KEY3 triggers key expansion
 - DATA_IN0 (0x12)
 - DATA_IN1 (0x13)
 - DATA_IN2 (0x14)
 - DATA_IN3 (0x15)
 - DATA_OUT0(0x22)
 - DATA_OUT1(0x23)

- DATA_OUT2(0x24)
- DATA_OUT3(0x25)
- (optionally IRQ_CLEAR at next slot)

This maps directly to the addresses you suggested but expands them into four-word slots for key/data.

4. Compact single-address variant (auto-increment)

- Behavior:
 - Writes to KEY_ADDR (0x02) are assembled into KEY word 0..3 in the order they arrive; the peripheral increments an internal key_write_index each write. When index reaches 4 (after the 4th write) it starts key expansion and sets KEY_READY when done.
 - Similar for DATA_IN (0x12): four writes fill the input block buffer and make it available for processing.
 - DATA_OUT (0x22) reads return the next word from a result buffer and auto-increment an output_index. After the 4th read the RESULT_VALID clears.
- Requirements & caveats:
 - Document the byte/word order (little-endian word packing) and that the nth write goes to word n.
 - Define behavior on overflow/extra writes (e.g., ignore when key is locked, or set ERROR).
 - Define behavior if START is asserted while you are mid-key or mid-data writes — reject or queue? Prefer: require key writes to complete before START=1.
 - Prevent races: make KEY_LOCK active immediately when set; if KEY_LOCK set, ignore subsequent KEY writes.

5. START semantics with your chosen map

- You said START=1 runs, START=0 resets. Clarify whether reset is soft (clear FIFOs / input buffers but preserve key) or hard (zeroize key/round-keys). I previously recommended soft reset + separate CLEAR_KEYS; confirm which you want.
- If you use auto-increment single-address mode, be sure START=1/0 behavior is well documented for in-flight partial-key/data writes.

6. STATUS and read behavior

- STATUS must expose KEY_READY and DONE. Add BUSY and ERROR fields if possible so software can avoid blocking reads and can detect if a result exists before reading DATA_OUT.
- For DATA_OUT reads: peripheral should only assert data_ready when a word is available; if software reads when no data present, either stall (do not assert data_ready) or set an UNDERFLOW error.

7. Software usage examples

- Explicit multi-register map:
 - write KEY0..KEY3
 - wait STATUS.KEY_READY
 - write DATA_IN0..3
 - set CTRL.START=1

- wait STATUS.DONE
 - read DATA_OUT0..3
- Auto-increment map:
 - write KEY four times to 0x02 (in documented order)
 - wait KEY_READY
 - write DATA four times to 0x12
 - set START=1
 - wait DONE
 - read DATA_OUT four times from 0x22

Recommendation

- Use the explicit multi-register mapping (Option A). It avoids ambiguities and simpler to test and reason about. Option B is compact but requires extra rules and careful documentation; it's ok if you prefer register economical design.

If you want, I will:

- Produce the Verilog register-decode skeleton + small FSM for the explicit map (Option A) implementing:
 - KEY0..KEY3 write handling & key expansion trigger,
 - DATA_IN0..3 input buffer,
 - single-result output registers DATA_OUT0..3,
 - CTRL.START reset semantics (pick soft reset or hard zeroize — tell me which),
 - STATUS bits and DATA_OUT read handshake.
- Or produce the compact auto-increment implementation (Option B) with the necessary rules and edge-case handling.

Which implementation do you want me to generate? Also confirm whether START==0 should be soft-reset (preserve key) or hard-reset (zeroize key).